1. INTRODUCTION

This project is testing validation of Deterministic Networking (DetNet) built up over five stages on a Linux test machine. DetNet is a set of IETF standards for carrying traffic that cannot tolerate packet loss or unpredictable delay, such as industrial control and power grid protection, across an ordinary packet network.

1.1 Background on DetNet

IP and Ethernet deliver traffic best effort, with no guarantee on bandwidth, latency, or jitter. Over thirty years, both added priority queuing on top: differentiated services at Layer 3, priority at Layer 2. Higher-priority packets go first, but exact arrival is still not promised. For most traffic, web, file transfer, and email, that is the right and inexpensive trade, since a packet a few milliseconds late is invisible.

It fails for traffic where delay and jitter change the service, so a late or lost packet is a real failure rather than an inconvenience. Industrial motion control, programmable logic controllers, professional audio, vehicles, and avionics all need a control loop that closes on time every cycle. These ran on purpose-built analog and digital networks precisely for that tight timing, in small constrained networks, not the global internet. Commodity Ethernet then became fast and cheap, making commercial packet technology an attractive replacement. TSN and DetNet were designed to serve these constrained environments.

This is not a contest over which technology is best, since IP and Ethernet overlap. IP is by far the most prevalent packet networking, the Layer 3 with global reach. Ethernet is usually the Layer 2 beneath it, either VLANs multiplexed on a link or a full bridged network, and at the edge in homes and small businesses bridging dominates the LAN. TSN (IEEE 802.1) came first as a Layer 2, single-subnet toolbox: time synchronization, scheduled gates, and reliability. DetNet (IETF) extends the same guarantees across routed IP and MPLS, often over TSN subnets, where Ethernet is not available or costs more than IP.

Reliability takes the same form at both layers. TSN uses FRER, Frame Replication and Elimination for Reliability: duplicate frames over disjoint paths, then delete the duplicates. DetNet uses PREOF, the Packet Replication, Elimination, and Ordering Function, for the identical job at Layer 3. They are one idea at two layers, so learning FRER first makes PREOF straightforward.

2. WORK PLAN

This document records the study and hands-on validation work carried out during the internship on deterministic networking, specifically the two service protection mechanisms Frame Replication and Elimination for Reliability (FRER) and the Packet Replication, Elimination, and Ordering Functions (PREOF).
Objective 1: Document any lessons learned when repeating DNT provided tests.

Objective 2: Create, test and publish, a munet topology to support FRER using bridges only, with no routers. Should include basic automated tests using mutest.
Identify the topology for this objective.
Objective 3: Create, test and publish, a munet topology that supports PREOF using static routers, with no FRR. Should include basic automated tests using mutest
Objective 4: Create, test and publish, a munet topology that adds dynamic routing with FRR to the PREOF test.
Objective 5: Create, test and publish, a munet topology that uses FRR to support Ethernet bridging over MPLS and demonstrate FRER running over the topology.

3. ENVIRONMENT AND PREREQUISITES

- Host: single Linux VM, Ubuntu 26.04 LTS (meets DNT's minimum kernel requirement of 4.20+)
- FRR (FRRouting): built from source, v10.8.0-dev, via the FRR Developer's Guide build path; validated against FRR's own OSPF topotest suite (ospf_topo1 plus the full ospf* suite)
- libyang: v3.13.6, built from source (FRR's YANG/NETCONF dependency, distro-packaged versions are below FRR's required minimum)
- munet: LabN's own topology orchestration tool (PyPI v0.17.3), installed in an isolated Python venv; builds the emulated topologies (network-namespace and podman container nodes)
- mutest: munet's companion test-writing layer (send/expect step functions), used for scripted topology validation
- DNT (Dependable Networking Toolkit): Ericsson Research, public GitHub repo, built from source; provides PRF/PEF for Layer 2 FRER (802.1CB) and Layer 3 PREOF (MPLS over UDP/IP)
- tshark / Wireshark: packet capture and protocol decode, used to confirm replication and elimination are actually running, not just forwarding
- podman: container runtime backing munet's container-node topologies
- pytest, scapy: Python test dependencies for FRR's topotest suite

4. OBJECTIVE 1

Learn about DNT and the test provided, then repeat the test

This objective repeated DNT's own published test, to confirm the toolkit works and to establish what valid evidence of DetNet redundancy looks like before building any topology of our own. Two of DNT's getting-started scenarios were reproduced on the lab VM: scenario_tsn, Layer 2 FRER using a VLAN tag plus an 802.1CB Redundancy Tag, and scenario_tsn_over_detnet, Layer 3 PREOF inside an MPLS-over-UDP pseudowire on port 6635. Both ran clean, 10 of 10 at 0% loss with no duplicates, and each is backed by a four vantage capture set taken from a single ping burst: one plain untagged copy entering at the talker, two differentiated copies crossing the two paths carrying the identical sequence number, and one fully decapsulated copy delivered at the listener. That set is the standard of proof every later objective follows, since a clean ping on its own would look identical if only one path were working. Two findings carried forward into everything after it. Capturing on two DNT-bound interfaces at once perturbs the tool into over-replicating, so every later capture is taken from a vantage that runs no DNT. And DNT's MPLS label rides inside a plain UDP/IP tunnel and exists only to tell the replicated copies apart, so the routers in the middle of the network need plain IP reachability and never an MPLS control plane.

Steps to achieve Objective 1:

- Step1. Study DNT's README and its scenario_tsn_over_detnet getting-started scenario in depth.
  - DNT's GitHub repository includes a top-level README (what the tool is, how to install it) and a "getting started" folder containing about ten worked example configurations, called scenarios, each one demonstrating a different feature. https://github.com/EricssonResearch/dnt/tree/main
- Step 2. Select the two scenarios that can be validated
  - Scenario 1: scenario_tsn (Layer 2, IEEE 802.1CB FRER): dnt/getting_started/scenario_tsn/README.md at main · EricssonResearch/dnt
  - Scenario 2: scenario_tsn_over_detnet (PREOF over an MPLS-over-UDP pseudowire): https://github.com/EricssonResearch/dnt/blob/main/getting_started/scenario_tsn_over_detnet/README.md
- Step 3. Prepare the environment for these test scenarios
  - Base machine: a Linux machine (this lab uses a single Ubuntu VM) meeting DNT's minimum kernel version, 4.20 or newer. Any mainstream Linux distro from the last several years satisfies this.
  - Install the build dependencies. DNT is written in C and has to be compiled before it can run, so the compiler toolchain has to be installed first, using the commands below:
    - ```
      # Ubuntu/Debian
      sudo apt install build-essential

      # Fedora/RHEL
      sudo dnf groupinstall @development-tools @development-libraries
      ```
  - Clone the repository. DNT's source code is public on GitHub. "Cloning" downloads a full copy of the repository, code plus history, onto the local machine, then cd moves the terminal into that new folder:
    - ```
      git clone https://github.com/EricssonResearch/dnt.git
      cd dnt
      ```
  - Build DNT. Running make reads the repo's build instructions and compiles the C source into a runnable program called dnt. sudo make install is an optional extra step that copies that program to a standard system location (like /usr/local/bin) so it can be run from any directory just by typing dnt, which is what lets the getting-started scripts call dnt dnt.ini directly instead of a full file path:
    - ```
      make
      sudo make install
      ```
      ```

    - Move into the specific getting-started scenario folder. Each scenario lives in its own subfolder under getting_started/, with its own config files (dnt.ini, or nxp1.ini/nxp2.ini) and its own env.sh:
        - ```
          cd getting_started/scenario_tsn
          # or, for the second scenario:
          cd getting_started/scenario_tsn_over_detnet
          ```
- Get root privileges. From inside that folder, access is needed for the rest of setup, since building the virtual test network and capturing raw packets both require it. This will make a new root shell:
    - `sudo -s`
- Build the virtual test network. The scenario folder's env.sh script automatically builds a small, disposable, self-contained virtual network using a Linux kernel feature called network namespaces: a way to run several completely isolated "mini networks" on one physical machine, each with its own interfaces, addresses, and routing table.
- Use 4 terminal windows or tmux panes each in the same directory then run the following below to begin this. Terminal #4 is for extra cmds and for organization
    - ```
      source env.sh
      ```

Network Topology for scenario 1:

The topology used to validate this scenario is a simple linear topology. Here, a talker host is connected to nxp1, and a listener connects to Nxp2, and nxp1 and nxp2 are connected through 2 point-to-point interfaces.

Talker and Listener are standard TSN/DetNet terms: Talker sends, Listener receives. DNT just reuses that naming.

nxp1 and nxp2 aren't running a special stack, just the DNT program itself, which does the replication and elimination.

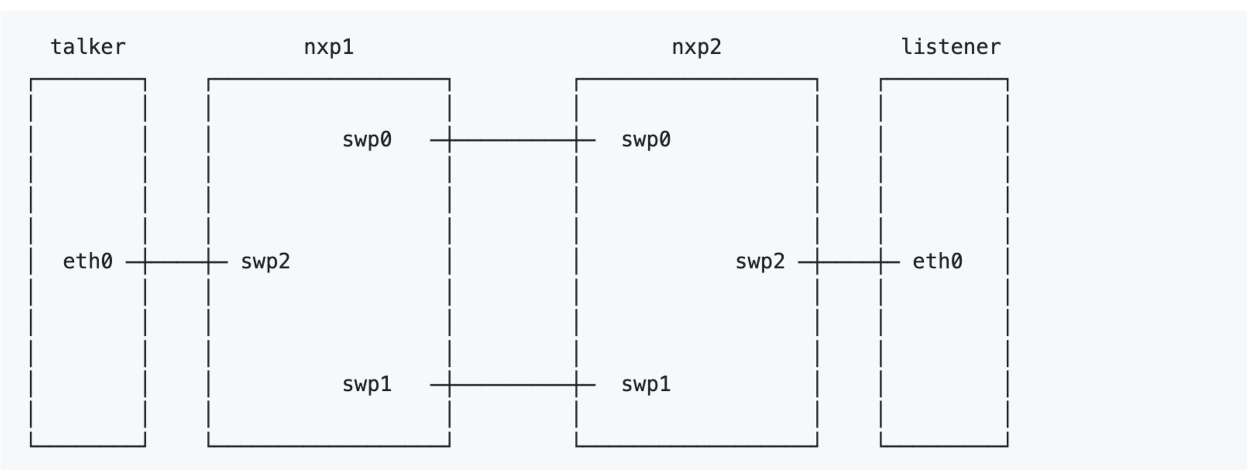


Fig 1: Scenario-1 topology

Scenario 1 Commands

- Following are the commands used to validate Scenario 1:
  - terminal 1: start DNT on nxp1 (loads its config file, dnt.ini)
    - `nxp1 dnt dnt.ini`
  - **terminal 2: start DNT on nxp2 (same config file, this scenario is symmetric)**
    - `nxp2 dnt dnt.ini`
  - terminal 3: Test connectivity between talker and listener
    - `talker ping -c 4 10.0.0.2`
  - You do not need to modify or paste this interface config it is already built in

```
[interfaces]
uni = eth iface=swp2
uni:streams = compound_uni
nni1 = eth iface=swp0
nni1:streams = member1_nni
nni2 = eth iface=swp1
nni2:streams = member2_nni

[objects]
Repl = Replicate
Gen = SeqGen InitSeqStart=0
Elim = SeqRcvy frerSeqRcvyHistoryLength=200

[streams]
compound_uni:packet = eth, cvlan
compound_uni:match = cvlan vid=0
compound_uni:actions = Gen, after cvlan add rtag, Repl Repl-member1 Repl-
member2

Repl-member1 = edit cvlan.vid=100, send nni1
Repl-member2 = edit cvlan.vid=200, send nni2

member1_nni:packet = eth, cvlan, rtag
member1_nni:match = cvlan vid=100
member1_nni:actions = readseq rtag, Elim Elim-compound

member2_nni:packet = eth, cvlan, rtag
member2_nni:match = cvlan vid=200
member2_nni:actions = readseq rtag, Elim Elim-compound

Elim-compound = del rtag, del cvlan, send uni
```

Steps to validate this scenario 1 from the shell

- Run this on one of the interfaces (swp0 or swp1), since that is the link carrying DNT's replicated, tagged traffic between nxp1 and nxp2, not the plain traffic on the UNI side.
- On nxp1, start the capture on the interface (swp0 or swp1):

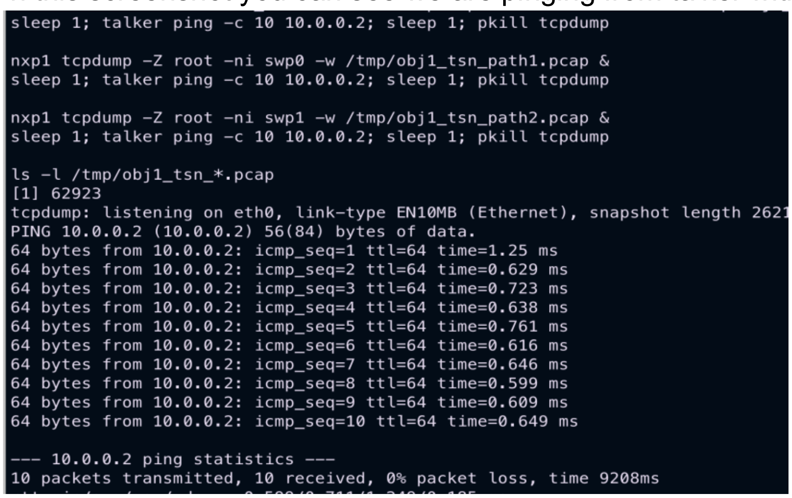
        - `tshark -l -O 'ieee8021cb' -i swp0 | grep -e 802 -e "10.0.0.1"`
    - On the talker CLI, generate traffic:
        - `talker ping -c 4 10.0.0.2`

    - Return to nxp1 and stop the capture:
        - `ctrl-c`
    - Return to nxp1 and stop the capture:
        - `ctrl-c`
- What to look for: each row shows an 802.1Q VLAN tag (ID 100 on one path, 200 on the other) and an 802.1CB Redundancy Tag whose SEQ counts up by one per packet.

Life of a Packet

Application Connectivity

In this screenshot you can see we are pinging from talker with 10 counts to listener



```
sleep 1; talker ping -c 10 10.0.0.2; sleep 1; pkill tcpdump

nxp1 tcpdump -Z root -ni swp0 -w /tmp/obj1_tsn_path1.pcap &
sleep 1; talker ping -c 10 10.0.0.2; sleep 1; pkill tcpdump

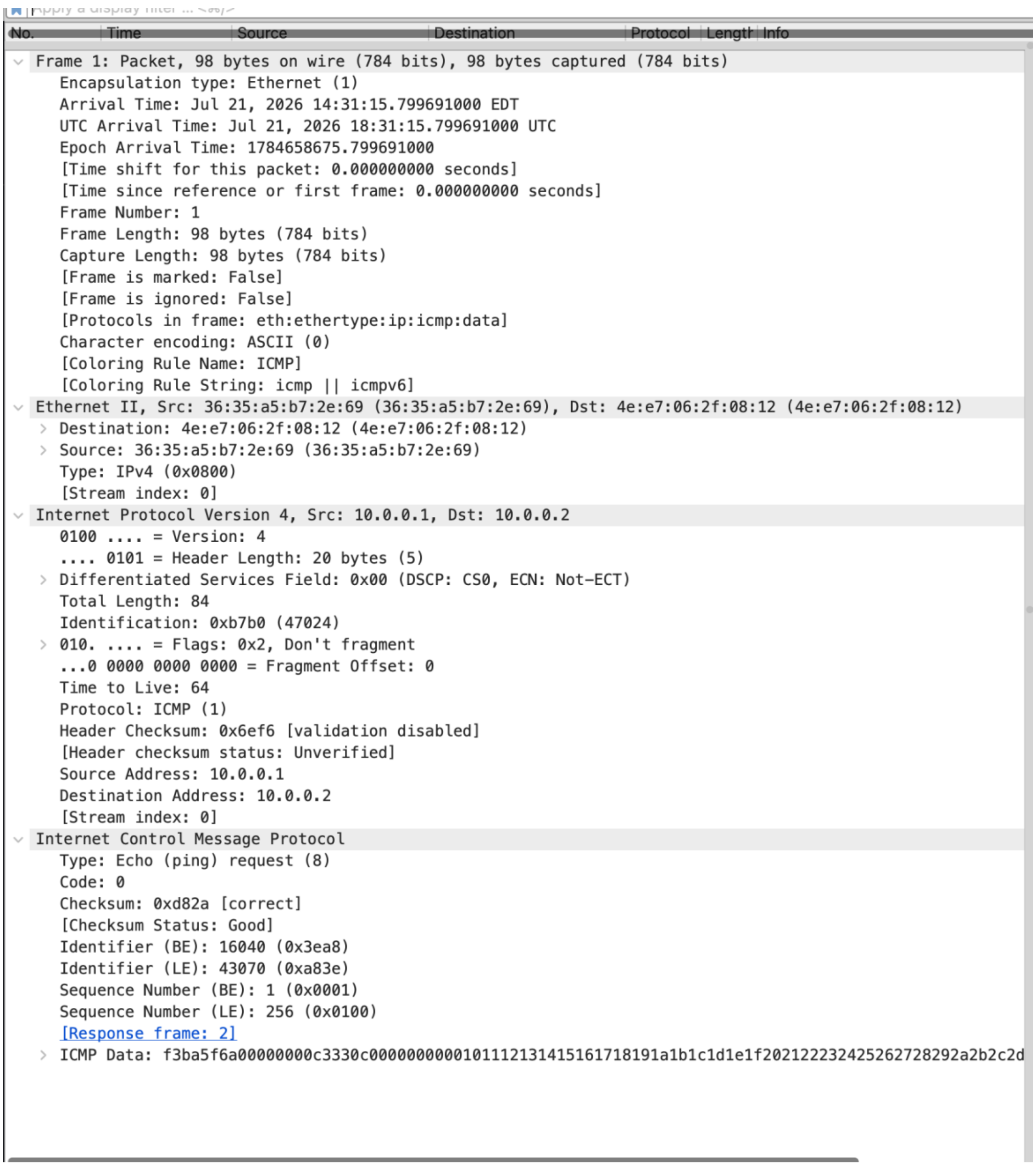
nxp1 tcpdump -Z root -ni swp1 -w /tmp/obj1_tsn_path2.pcap &
sleep 1; talker ping -c 10 10.0.0.2; sleep 1; pkill tcpdump

ls -l /tmp/obj1_tsn_*.pcap
[1] 62923
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 2621
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.25 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.629 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.723 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.638 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.761 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.616 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.646 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.599 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.609 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.649 ms

--- 10.0.0.2 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9208ms
```

Packet capture on Eth0-Swp2 (Talker):

- Here is a packet captured on Eth0-Swp2 as you can see, it is a regular ICMP packet with no DNT encapsulation. The VLAN ID matches the member details as listed in the config above




```
Frame 1: Packet, 98 bytes on wire (784 bits), 98 bytes captured (784 bits)
    Encapsulation type: Ethernet (1)
    Arrival Time: Jul 21, 2026 14:31:15.799691000 EDT
    UTC Arrival Time: Jul 21, 2026 18:31:15.799691000 UTC
    Epoch Arrival Time: 1784658675.799691000
    [Time shift for this packet: 0.000000000 seconds]
    [Time since reference or first frame: 0.000000000 seconds]
    Frame Number: 1
    Frame Length: 98 bytes (784 bits)
    Capture Length: 98 bytes (784 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:icmp:data]
    Character encoding: ASCII (0)
    [Coloring Rule Name: ICMP]
    [Coloring Rule String: icmp || icmpv6]
Ethernet II, Src: 36:35:a5:b7:2e:69 (36:35:a5:b7:2e:69), Dst: 4e:e7:06:2f:08:12 (4e:e7:06:2f:08:12)
  > Destination: 4e:e7:06:2f:08:12 (4e:e7:06:2f:08:12)
  > Source: 36:35:a5:b7:2e:69 (36:35:a5:b7:2e:69)
    Type: IPv4 (0x0800)
    [Stream index: 0]
Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 84
    Identification: 0xb7b0 (47024)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: ICMP (1)
    Header Checksum: 0x6ef6 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.0.0.1
    Destination Address: 10.0.0.2
    [Stream index: 0]
Internet Control Message Protocol
    Type: Echo (ping) request (8)
    Code: 0
    Checksum: 0xd82a [correct]
    [Checksum Status: Good]
    Identifier (BE): 16040 (0x3ea8)
    Identifier (LE): 43070 (0xa83e)
    Sequence Number (BE): 1 (0x0001)
    Sequence Number (LE): 256 (0x0100)
    [Response frame: 2]
  > ICMP Data: f3ba5f6a00000000c3330c0000000000101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d
```

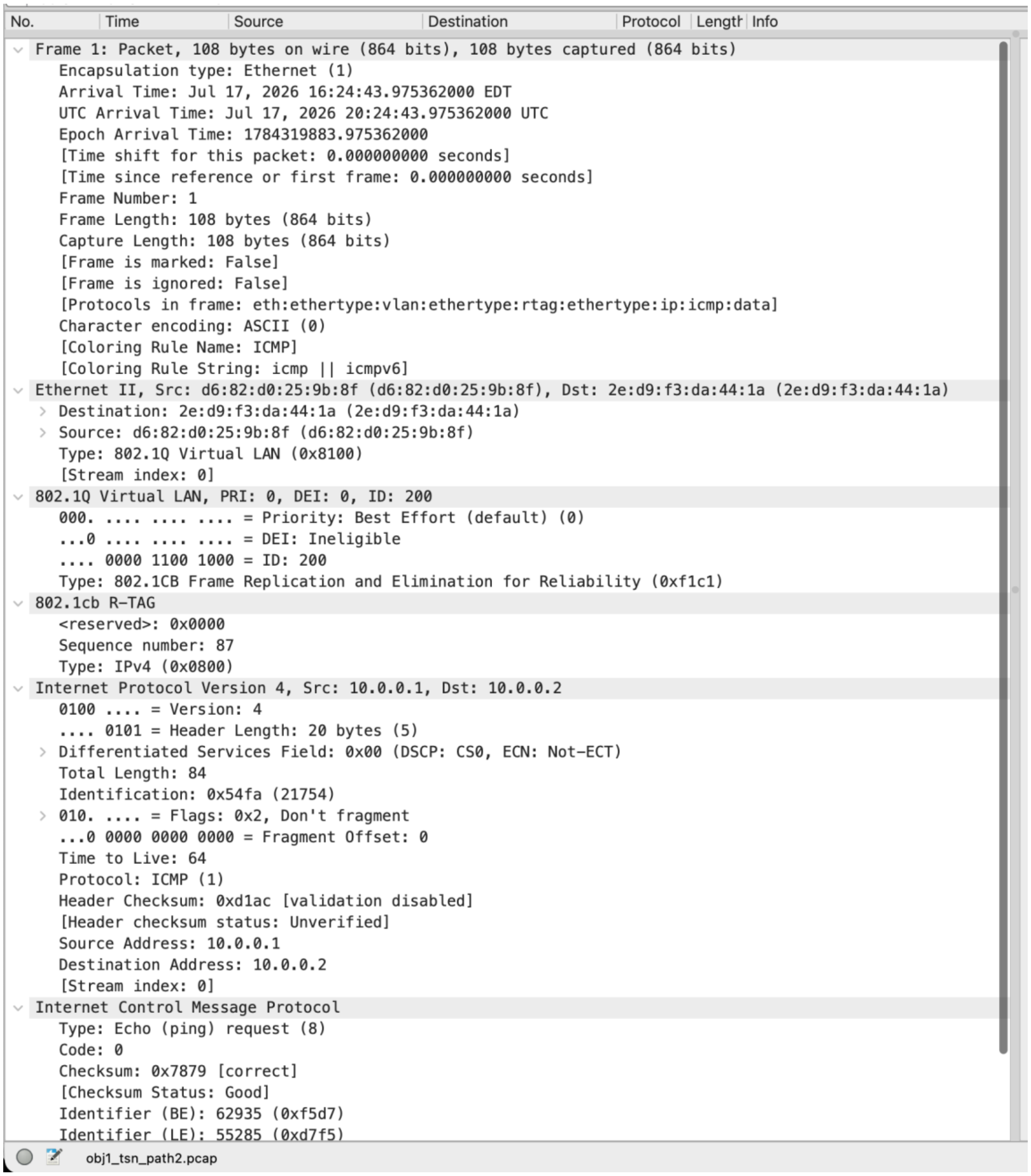
Packet capture on Swp0-Swp0:

- Here is a packet captured on Swp0-Swp0 as you can see, one packet is being sent from talker to listener encapsulated with the VLAN ID = 100 as well as the R-Tag = 77. Same thing here details such as the VLAN ID are also matching the member's ID from the interface config

```
No.        Time        Source        Destination        Protocol  Length  Info
Frame 1: Packet, 108 bytes on wire (864 bits), 108 bytes captured (864 bits)
    Encapsulation type: Ethernet (1)
    Arrival Time: Jul 17, 2026 16:24:32.809400000 EDT
    UTC Arrival Time: Jul 17, 2026 20:24:32.809400000 UTC
    Epoch Arrival Time: 1784319872.809400000
    [Time shift for this packet: 0.000000000 seconds]
    [Time since reference or first frame: 0.000000000 seconds]
    Frame Number: 1
    Frame Length: 108 bytes (864 bits)
    Capture Length: 108 bytes (864 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:vlan:ethertype:rtag:ethertype:ip:icmp:data]
    Character encoding: ASCII (0)
    [Coloring Rule Name: ICMP]
    [Coloring Rule String: icmp || icmpv6]
Ethernet II, Src: d6:82:d0:25:9b:8f (d6:82:d0:25:9b:8f), Dst: 2e:d9:f3:da:44:1a (2e:d9:f3:da:44:1a)
  > Destination: 2e:d9:f3:da:44:1a (2e:d9:f3:da:44:1a)
  > Source: d6:82:d0:25:9b:8f (d6:82:d0:25:9b:8f)
    Type: 802.1Q Virtual LAN (0x8100)
    [Stream index: 0]
802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 100
    000. .... .... .... = Priority: Best Effort (default) (0)
    ...0 .... .... .... = DEI: Ineligible
    .... 0000 0110 0100 = ID: 100
    Type: 802.1CB Frame Replication and Elimination for Reliability (0xf1c1)
802.1cb R-TAG
    <reserved>: 0x0000
    Sequence number: 77
    Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 84
    Identification: 0x408c (16524)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: ICMP (1)
    Header Checksum: 0xe61a [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.0.0.1
    Destination Address: 10.0.0.2
    [Stream index: 0]
Internet Control Message Protocol
    Type: Echo (ping) request (8)
    Code: 0
    Checksum: 0x5606 [correct]
    [Checksum Status: Good]
    Identifier (BE): 62930 (0xf5d2)
    Identifier (LE): 54005 (0xd2f5)
```

Packet capture on Swp1-Swp1:

- Here is a packet capture from Swp1-Swp1. It is identical to the packet capture of Swp0-Swp0 except the VLAN ID and the R-TAG are different



Packet capture on Swp2-Eth0 (Listener):

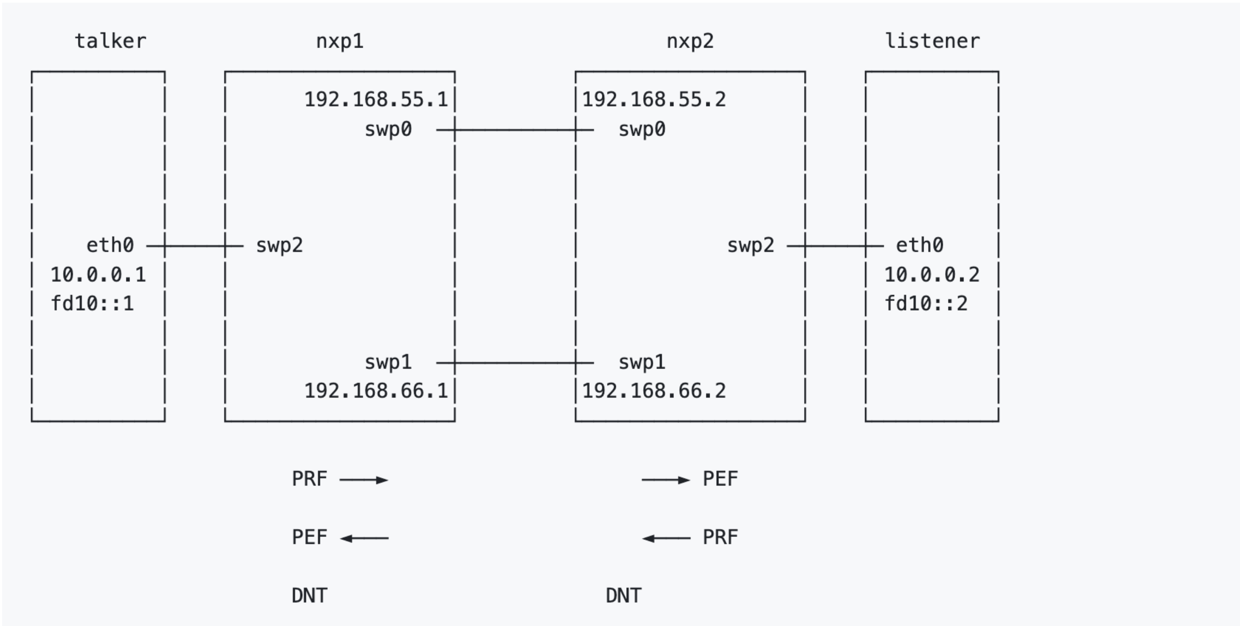
- Here is a packet capture from Listener as you can see listener recieves an ICMP packet stripped of the VLAN ID and the R Tag

```
Frame 1: Packet, 98 bytes on wire (784 bits), 98 bytes captured (784 bits)
    Encapsulation type: Ethernet (1)
    Arrival Time: Jul 17, 2026 16:24:21.560581000 EDT
    UTC Arrival Time: Jul 17, 2026 20:24:21.560581000 UTC
    Epoch Arrival Time: 1784319861.560581000
    [Time shift for this packet: 0.000000000 seconds]
    [Time since reference or first frame: 0.000000000 seconds]
    Frame Number: 1
    Frame Length: 98 bytes (784 bits)
    Capture Length: 98 bytes (784 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:icmp:data]
    Character encoding: ASCII (0)
    [Coloring Rule Name: ICMP]
    [Coloring Rule String: icmp || icmpv6]
Ethernet II, Src: d6:82:d0:25:9b:8f (d6:82:d0:25:9b:8f), Dst: 2e:d9:f3:da:44:1a (2e:d9:f3:da:44:1a)
  > Destination: 2e:d9:f3:da:44:1a (2e:d9:f3:da:44:1a)
  > Source: d6:82:d0:25:9b:8f (d6:82:d0:25:9b:8f)
    Type: IPv4 (0x0800)
    [Stream index: 0]
Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 84
    Identification: 0x3222 (12834)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: ICMP (1)
    Header Checksum: 0xf484 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.0.0.1
    Destination Address: 10.0.0.2
    [Stream index: 0]
Internet Control Message Protocol
    Type: Echo (ping) request (8)
    Code: 0
    Checksum: 0x99d8 [correct]
    [Checksum Status: Good]
    Identifier (BE): 62925 (0xf5cd)
    Identifier (LE): 52725 (0xcdf5)
    Sequence Number (BE): 1 (0x0001)
    Sequence Number (LE): 256 (0x0100)
    [Response frame: 2]
  > ICMP Data: 758f5a6a00000000d18b08000000000000101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c…
```

Scenario 2 Commands:

- **Terminal 1: start DNT on nxp1**
  - `nxp1 dnt nxp1.ini`
- Terminal 2: start DNT on nxp2 (its own config, since the two routers have different IPs)
  - `nxp2 dnt nxp2.ini`
- Terminal 3: Test connectivity between talker and listener
  - `talker ping -c 4 10.0.0.2`

Network Topology for scenario 2:

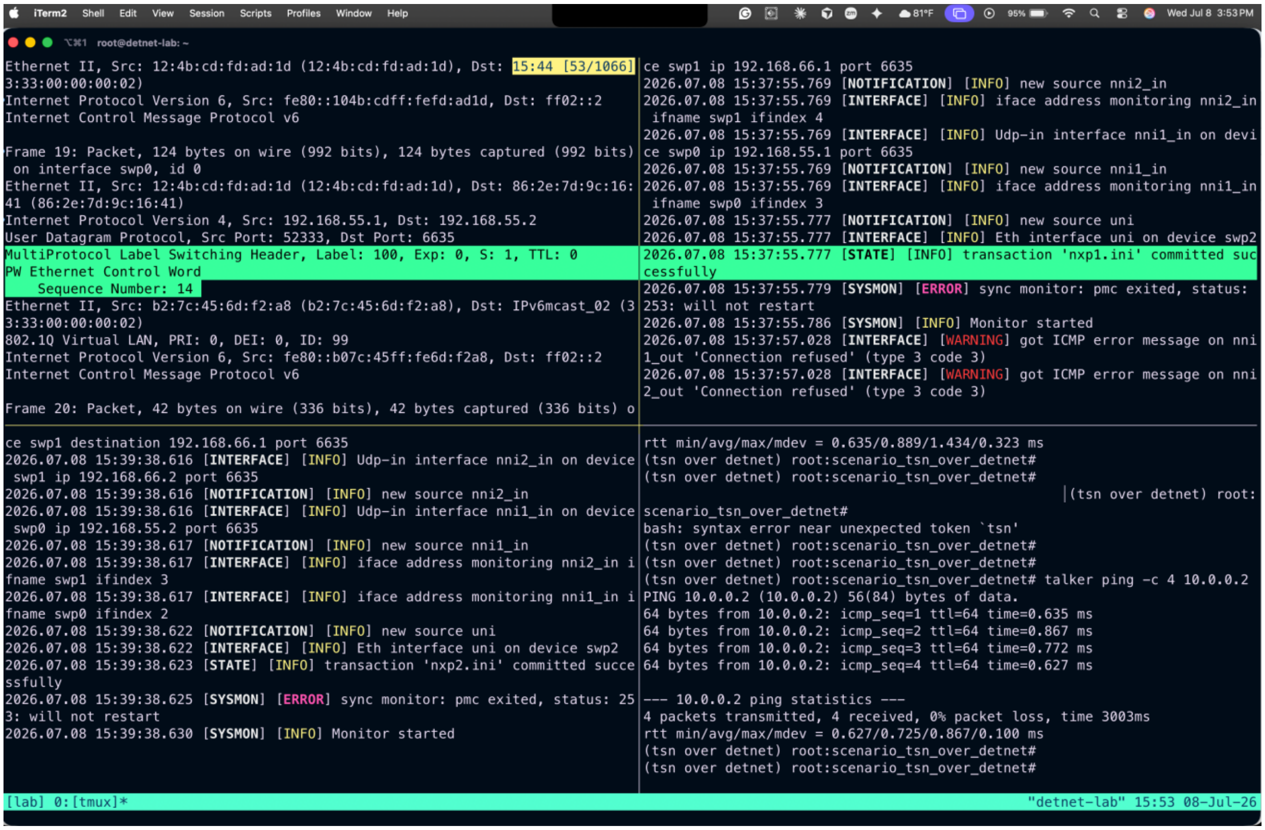


Steps to validate this scenario 2 from the shell

- Run tshark on nxp1 swp0
  - `nxp1 tshark -i swp0`
- Then on the talker’s CLI ping to the listener to generate traffic
  - `talker ping -c 4 10.0.0.2`
- Return to nxp1 and stop the capture:
  - `ctrl-c`

- What to look for, reading from the outside in: a plain Ethernet/IPv4/UDP envelope carrying traffic between the two routers' IPs over UDP port 6635. Inside it, an MPLS label (100 or 200, identifying the replicated copy) followed by a DetNet Control Word (d-CW, sitting in the pseudowire control-word position) whose sequence number counts up per packet. Inside that, the original frame the talker sent (VLAN, IPv4, ICMP), unchanged.

I highlight MPLS being used as well as the sequence number, proving that PREOF was being used as stated in the RFC. The test can be found below.
https://github.com/EricssonResearch/dnt/blob/main/getting_started/scenario_tsn_over_detnet/README.md



Life of a Packet

Application Connectivity

In this screenshot you can see we are pinging from talker with 10 counts to listener

```
(tsn over detnet) root:scenario_tsn_over_detnet# talker ping -c 10 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.629 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.562 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.517 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.616 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.563 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.825 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.602 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.379 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.526 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.615 ms

--- 10.0.0.2 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9256ms
rtt min/avg/max/mdev = 0.379/0.583/0.825/0.106 ms
(tsn over detnet) root:scenario_tsn_over_detnet# sleep 1
(tsn over detnet) root:scenario_tsn_over_detnet# pkill tcpdump
```

Packet capture on Eth0-Swp2:

- Here is a packet captured on Eth0-Swp2, the UNI link between the talker and nxp1. It is a regular ICMP packet with no DNT encapsulation: plain Ethernet, IPv4 (10.0.0.1 to 10.0.0.2), ICMP, untagged.

```
No.        Time        Source         Destination        Protocol  Length  Info
Frame 1: Packet, 98 bytes on wire (784 bits), 98 bytes captured (784 bits)
    Encapsulation type: Ethernet (1)
    Arrival Time: Jul 23, 2026 12:32:36.774000000 EDT
    UTC Arrival Time: Jul 23, 2026 16:32:36.774000000 UTC
    Epoch Arrival Time: 1784824356.774000000
    [Time shift for this packet: 0.000000000 seconds]
    [Time since reference or first frame: 0.000000000 seconds]
    Frame Number: 1
    Frame Length: 98 bytes (784 bits)
    Capture Length: 98 bytes (784 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:icmp:data]
    Character encoding: ASCII (0)
    [Coloring Rule Name: ICMP]
    [Coloring Rule String: icmp || icmpv6]
Ethernet II, Src: aa:cf:be:54:3a:c0 (aa:cf:be:54:3a:c0), Dst: 5a:ea:ae:fa:07:d2 (5a:ea:ae:fa:07:d2)
  > Destination: 5a:ea:ae:fa:07:d2 (5a:ea:ae:fa:07:d2)
  > Source: aa:cf:be:54:3a:c0 (aa:cf:be:54:3a:c0)
    Type: IPv4 (0x0800)
    [Stream index: 0]
Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 84
    Identification: 0x84a8 (33960)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: ICMP (1)
    Header Checksum: 0xa1fe [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.0.0.1
    Destination Address: 10.0.0.2
    [Stream index: 0]
Internet Control Message Protocol
    Type: Echo (ping) request (8)
    Code: 0
    Checksum: 0xf605 [correct]
    [Checksum Status: Good]
    Identifier (BE): 38828 (0x97ac)
    Identifier (LE): 44183 (0xac97)
    Sequence Number (BE): 1 (0x0001)
    Sequence Number (LE): 256 (0x0100)
    [Response frame: 2]
  > ICMP Data: 2442626a0000000019cd0b0000000000101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d
```

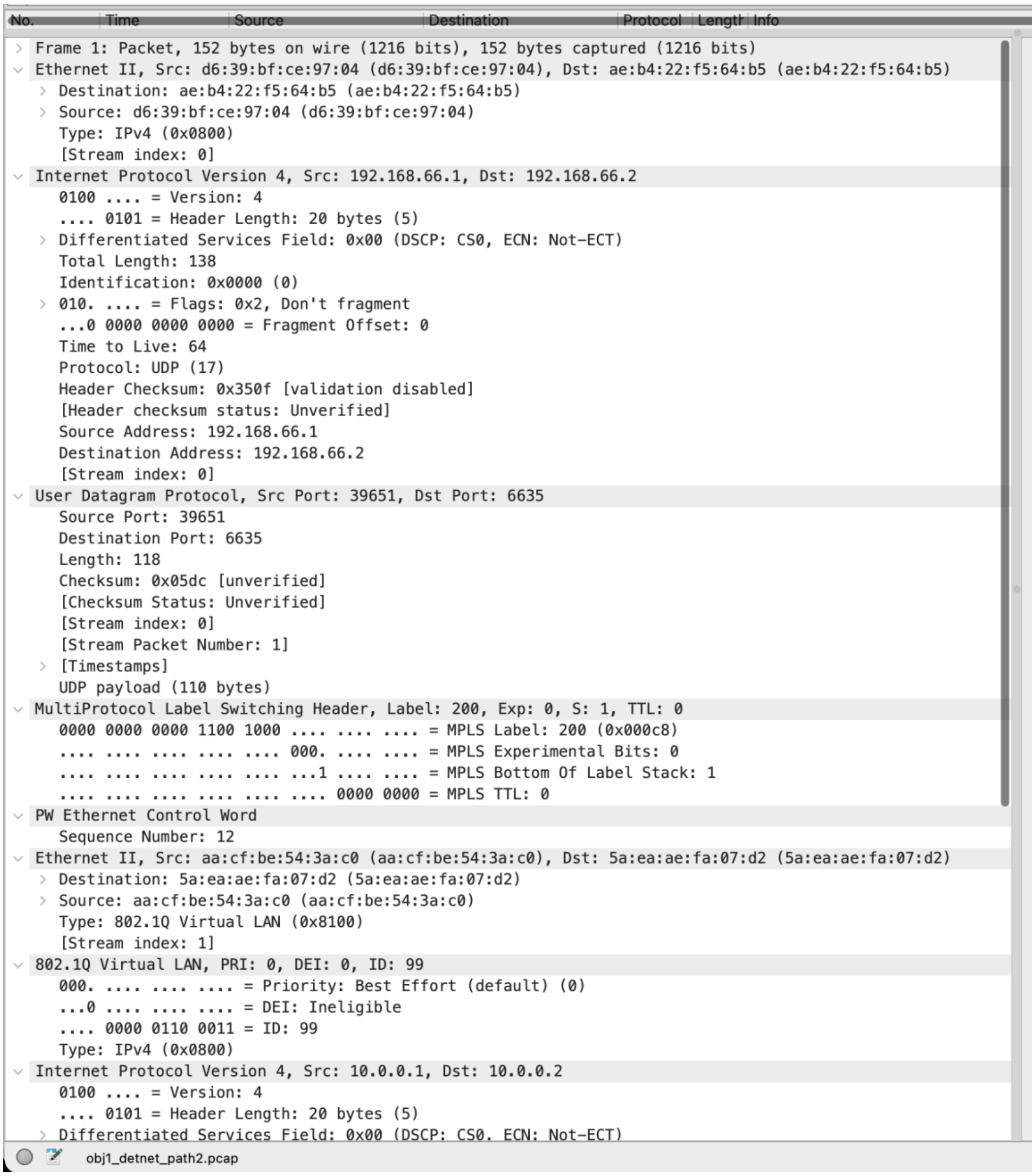
Packet capture on Swp0-Swp0:

- Here is a packet captured on Swp0-Swp0, Path 1 between nxp1 and nxp2. One replicated copy of the talker to listener traffic is sent here, encapsulated by DNT's PREOF: outer IPv4 192.168.55.1 to 192.168.55.2, UDP destination port 6635, MPLS label = 100, and a PW Ethernet Control Word carrying the DetNet sequence number. Inside the pseudowire is the original frame: inner VLAN ID (cvlan) = 99 and the host ICMP 10.0.0.1 to 10.0.0.2. The MPLS label 100 matches member1 in the config (member1:match = mpls label=100), and the inner cvlan 99 confirms the packet belongs to the stream.

```
No.          Time            Source                 Destination            Protocol  Length  Info
> Frame 3: Packet, 152 bytes on wire (1216 bits), 152 bytes captured (1216 bits)
v Ethernet II, Src: ce:1a:96:54:48:19 (ce:1a:96:54:48:19), Dst: c6:cc:23:ed:3d:78 (c6:cc:23:ed:3d:78)
  > Destination: c6:cc:23:ed:3d:78 (c6:cc:23:ed:3d:78)
  > Source: ce:1a:96:54:48:19 (ce:1a:96:54:48:19)
    Type: IPv4 (0x0800)
    [Stream index: 0]
v Internet Protocol Version 4, Src: 192.168.55.1, Dst: 192.168.55.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 138
    Identification: 0x0000 (0)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: UDP (17)
    Header Checksum: 0x4b0f [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.55.1
    Destination Address: 192.168.55.2
    [Stream index: 0]
v User Datagram Protocol, Src Port: 51673, Dst Port: 6635
    Source Port: 51673
    Destination Port: 6635
    Length: 118
    Checksum: 0xefdb [unverified]
    [Checksum Status: Unverified]
    [Stream index: 0]
    [Stream Packet Number: 2]
  > [Timestamps]
    UDP payload (110 bytes)
v MultiProtocol Label Switching Header, Label: 100, Exp: 0, S: 1, TTL: 0
    0000 0000 0000 0110 0100 .... .... .... = MPLS Label: 100 (0x00064)
    .... .... .... .... .... 000. .... .... = MPLS Experimental Bits: 0
    .... .... .... .... .... ...1 .... .... = MPLS Bottom Of Label Stack: 1
    .... .... .... .... .... .... 0000 0000 = MPLS TTL: 0
v PW Ethernet Control Word
    Sequence Number: 1
v Ethernet II, Src: aa:cf:be:54:3a:c0 (aa:cf:be:54:3a:c0), Dst: 5a:ea:ae:fa:07:d2 (5a:ea:ae:fa:07:d2)
  > Destination: 5a:ea:ae:fa:07:d2 (5a:ea:ae:fa:07:d2)
  > Source: aa:cf:be:54:3a:c0 (aa:cf:be:54:3a:c0)
    Type: 802.1Q Virtual LAN (0x8100)
    [Stream index: 2]
v 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 99
    000. .... .... .... = Priority: Best Effort (default) (0)
    ...0 .... .... .... = DEI: Ineligible
    .... 0000 0110 0011 = ID: 99
    Type: IPv4 (0x0800)
v Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
obj1_detnet_path1.pcap
```

Packet capture on Swp1-Swp1:

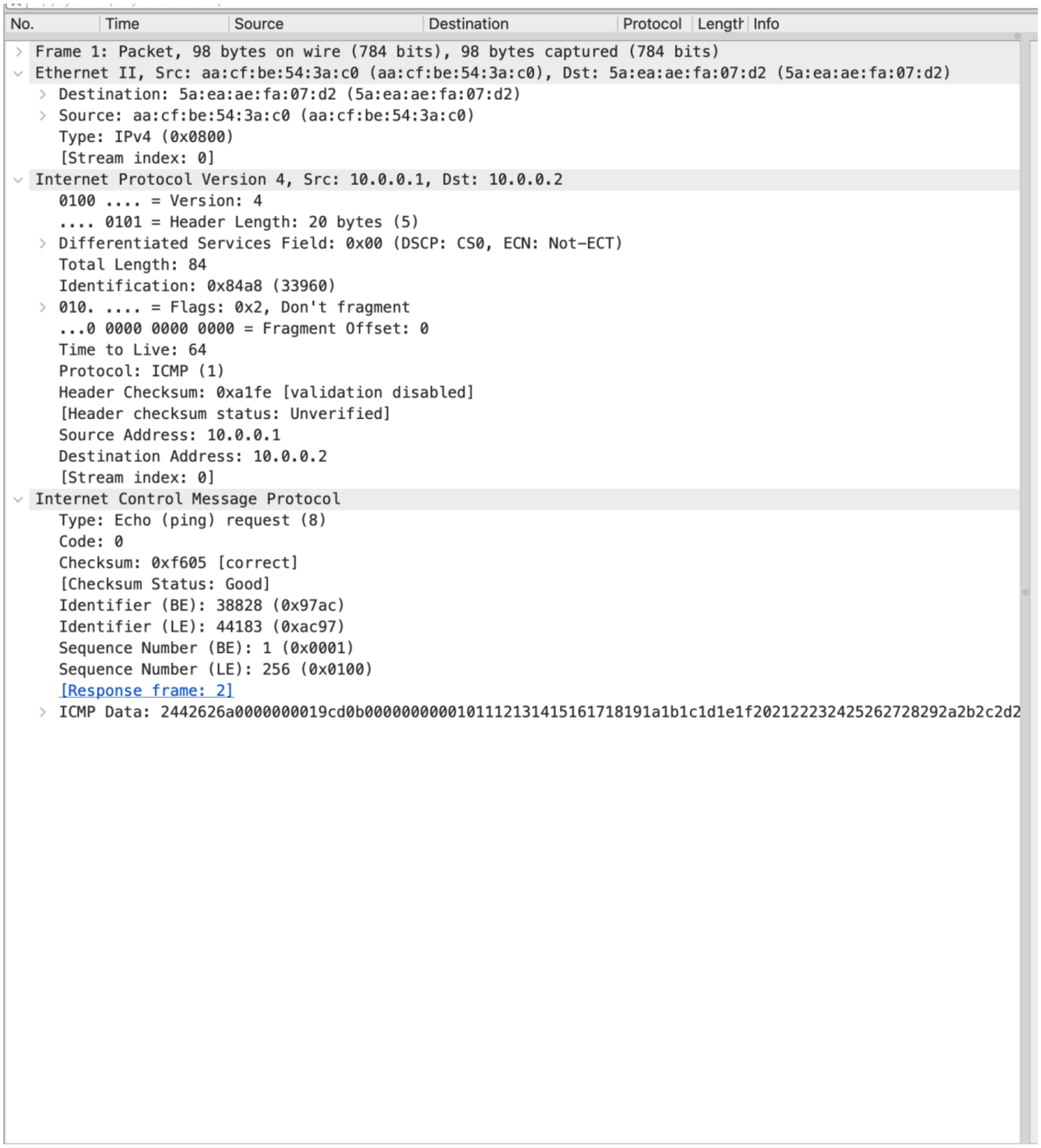
- Here is a packet captured on Swp1-Swp1, Path 2. It is identical in structure to the Swp0-Swp0 capture except the outer addresses are 192.168.66.1 to 192.168.66.2 and the MPLS label = 200, matching member2 in the config (member2:match = mpls label=200). The control word carries the same sequence number as the Path 1 copy for the same ping, and the inner cvlan 99 and host ICMP are unchanged.



```
No.   Time   Source   Destination   Protocol   Length   Info
> Frame 1: Packet, 152 bytes on wire (1216 bits), 152 bytes captured (1216 bits)
v Ethernet II, Src: d6:39:bf:ce:97:04 (d6:39:bf:ce:97:04), Dst: ae:b4:22:f5:64:b5 (ae:b4:22:f5:64:b5)
  > Destination: ae:b4:22:f5:64:b5 (ae:b4:22:f5:64:b5)
  > Source: d6:39:bf:ce:97:04 (d6:39:bf:ce:97:04)
    Type: IPv4 (0x0800)
    [Stream index: 0]
v Internet Protocol Version 4, Src: 192.168.66.1, Dst: 192.168.66.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 138
    Identification: 0x0000 (0)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: UDP (17)
    Header Checksum: 0x350f [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.66.1
    Destination Address: 192.168.66.2
    [Stream index: 0]
v User Datagram Protocol, Src Port: 39651, Dst Port: 6635
    Source Port: 39651
    Destination Port: 6635
    Length: 118
    Checksum: 0x05dc [unverified]
    [Checksum Status: Unverified]
    [Stream index: 0]
    [Stream Packet Number: 1]
  > [Timestamps]
    UDP payload (110 bytes)
v MultiProtocol Label Switching Header, Label: 200, Exp: 0, S: 1, TTL: 0
    0000 0000 0000 1100 1000 .... .... .... = MPLS Label: 200 (0x000c8)
    .... .... .... .... .... 000. .... .... = MPLS Experimental Bits: 0
    .... .... .... .... .... ...1 .... .... = MPLS Bottom Of Label Stack: 1
    .... .... .... .... .... .... 0000 0000 = MPLS TTL: 0
v PW Ethernet Control Word
    Sequence Number: 12
v Ethernet II, Src: aa:cf:be:54:3a:c0 (aa:cf:be:54:3a:c0), Dst: 5a:ea:ae:fa:07:d2 (5a:ea:ae:fa:07:d2)
  > Destination: 5a:ea:ae:fa:07:d2 (5a:ea:ae:fa:07:d2)
  > Source: aa:cf:be:54:3a:c0 (aa:cf:be:54:3a:c0)
    Type: 802.1Q Virtual LAN (0x8100)
    [Stream index: 1]
v 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 99
    000. .... .... .... = Priority: Best Effort (default) (0)
    ...0 .... .... .... = DEI: Ineligible
    .... 0000 0110 0011 = ID: 99
    Type: IPv4 (0x0800)
v Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
obj1_detnet_path2.pcap
```

Packet capture on Swp2-Eth0 (Listener):

- Here is a packet captured at the listener, the Swp2-Eth0 UNI link. The listener receives a plain ICMP packet, fully decapsulated: the MPLS label, UDP tunnel, PW control word, and inner cvlan 99 have all been stripped by nxp2's, which also eliminated the duplicate copy.



5. OBJECTIVE 2

Set up munet topology to support FRER using bridges only, with no routers

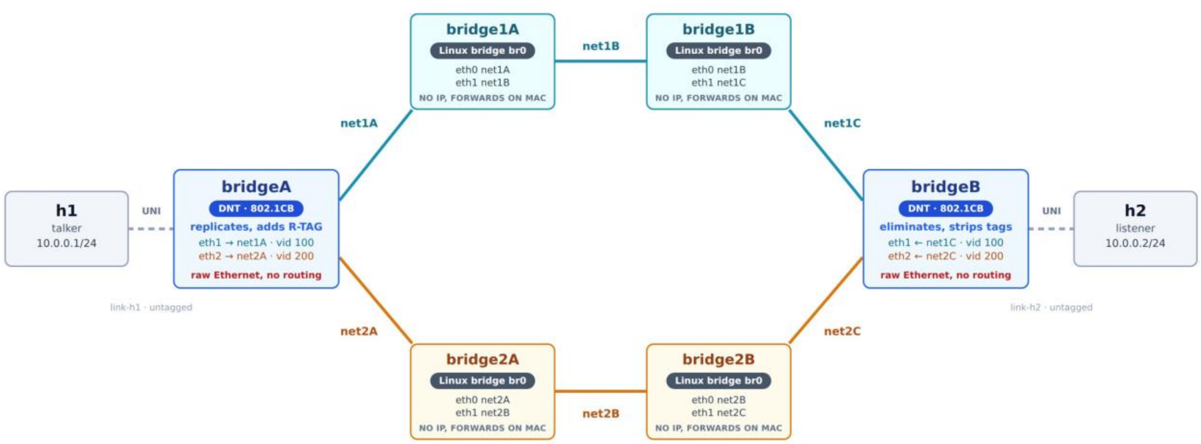
This objective built the lab's own FRER topology in munet instead of reusing DNT's supplied sandbox, and proved from a packet capture that replication and elimination really run across two independent paths. The topology is eight nodes and eight links: two end hosts, two edge nodes running DNT's replicate and eliminate functions, and two node-disjoint paths of two plain Linux bridges each. Holding the whole thing at Layer 2 with no routers isolates FRER so it can be validated on its own, before PREOF and dynamic routing are added in the objectives that follow. The result: ping returned 4 of 4 at 0% loss with no duplicates, and a capture taken from an inner bridge that runs no DNT showed every Path 1 frame carrying VLAN tag 100 and an 802.1CB R-TAG whose sequence number increments by one per frame, with Path 2 carrying VLAN 200. One failure along the way shaped the final design: placeholder Linux bridges at the edges formed a Layer 2 loop and a broadcast storm of over 2.6 million frames from a single ARP query, which vanished once real DNT replaced them, confirming that the edges must replicate deliberately rather than bridge blindly.

The goal is to build our own FRER topology in munet, not reuse DNT's sandbox from Objective 1, and prove from a packet capture that replication and elimination really run across two independent paths, not just that a ping succeeds.

Steps to Validate Objective 2:

Step 1. Identify the topology

FRER needs two node-disjoint paths and one device at each edge doing the replicate and eliminate work. Building it at Layer 2 with plain bridges and no routers isolates FRER, so it can be validated before PREOF (the Layer 3 equivalent) and FRR routing are added later.



The edge nodes are not ordinary bridges. A standard bridge runs Spanning Tree (STP) to keep a single active path, the opposite of what FRER needs. Bridge A deliberately sends the same frame out two interfaces. That is safe because Path 1 and Path 2 are separate networks to Bridge A, not one network with a redundant link, so there is no loop for STP to break.

Step 3. Prepare the environment.

- Install munet in an isolated Python venv so its dependencies do not collide with the system's:

  - ```
    python3 -m venv ~/lab/munet-venv
    ~/lab/munet-venv/bin/pip install munet
    ```

- **Symlink munet onto the system path so sudo munet resolves through the venv (the topology build needs root):**

  - ```
    sudo ln -sf ~/lab/munet-venv/bin/munet /usr/local/bin/munet sudo ln -sf
    ~/lab/munet-venv/bin/mutest /usr/local/bin/mutest
    ```

- **Confirm DNT (built in Objective 1) is present, since the edge nodes call it by absolute path expect /usr/local/bin/dnt or which ever directory you selected to install the enviroment:**

  - ```
    which dnt
    ```

Step 4. Build the topology and install DNT onto the edge nodes.

- One munet YAML file, three reusable node types (kinds):
- **host**: brings up its interface, assigns its address, and disables checksum and VLAN offload. Offload must be off because on veth links it leaves DNT's forwarded copies with unfinished checksums that the far host drops.
- **bridge**: the four inner nodes. Each makes a Linux bridge over its two interfaces and forwards transparently.
- **edge**: the two DNT nodes. Each brings up its three interfaces and launches DNT in the namespace. DNT owns the raw interfaces, so no bridge device is needed.
- Create a yaml file in your chosen directory, name it “munet”. Copy the munet config into the .yaml file.
- Run the following optional but will make things easier (create shell environments for the host, edge, and bridge nodes this requires tmux) without tmux you will need to use munet’s CLI to send commands to nodes:
    ```

- ```
  munet -c munet.yaml

  Generate traffic from Host A (the talker) to Host B (the listener):

  ping -c 4 10.0.0.2
  ```

- **If tmux is installed:**

  - ```
    munet -c munet.yaml --shell h1,h2,bridgeA,bridgeB,bridge1A
    ```

-
  - ```
    ping -c 4 10.0.0.2
    ```

Munet Config

```
Munet Config:
version: 1
topology:
  networks:
  - name: link-h1
  - name: net1A
  - name: net1B
  - name: net1C
  - name: net2A
  - name: net2B
  - name: net2C
  - name: link-h2

  nodes:
  - name: h1
    kind: host
    cmd: |
      ip addr add 10.0.0.1/24 dev eth0
      ip link set eth0 up
      bash
    connections:
    - to: link-h1

  - name: bridgeA
    kind: edge
    connections:
    - to: link-h1
    - to: net1A
    - to: net2A
```
```

```
  - name: bridge1A
    kind: bridge
    connections:
    - to: net1A
    - to: net1B

  - name: bridge1B
    kind: bridge
    connections:
    - to: net1B
    - to: net1C

  - name: bridge2A
    kind: bridge
    connections:
    - to: net2A
    - to: net2B

  - name: bridge2B
    kind: bridge
    connections:
    - to: net2B
    - to: net2C

  - name: bridgeB
    kind: edge
    connections:
    - to: link-h2
    - to: net1C
    - to: net2C

  - name: h2
    kind: host
    cmd: |
      ip addr add 10.0.0.2/24 dev eth0
      ip link set eth0 up
      bash
    connections:
    - to: link-h2

kinds:
- name: host
  cmd: |
    ip link set eth0 up
    ip addr add 10.0.0.1/24 dev eth0
    ethtool -K eth0 rxvlan off txvlan off tx off rx off

- name: bridge
```

```
  cmd: |
    ip link add name br0 type bridge
    ip link set eth0 master br0
    ip link set eth1 master br0
    ip link set br0 up

- name: edge
  cmd: |
    ip link set eth0 up; ip link set eth1 up;
    ip link set eth2 up
    /usr/local/bin/dnt dnt.ini
```

DNT config:

(the scenario_tsn dnt.ini, run on both edges). [interfaces] names the three ports; [objects] defines the replicate, sequence-generate, and recovery functions; [streams] is the FRER logic. On ingress, DNT adds a sequence number and the 802.1CB R-TAG, then replicates onto Path 1 (VLAN 100) and Path 2 (VLAN 200). On return it matches the VLAN, reads the sequence number, runs 802.1CB recovery to drop duplicates, strips the tags, and delivers one clean copy:

```
[interfaces]
uni = eth iface=eth0
uni:streams = compound_uni
nni1 = eth iface=eth1
nni1:streams = member1_nni
nni2 = eth iface=eth2
nni2:streams = member2_nni

[objects]
Repl = Replicate
Gen = SeqGen InitSeqStart=0
Elim = SeqRcvy frerSeqRcvyHistoryLength=200

[streams]
compound_uni:packet = eth, cvlan
compound_uni:match = cvlan vid=0
compound_uni:actions = Gen, after cvlan add rtag, writeseq rtag, Repl Repl-member1
Repl-member2

Repl-member1 = edit cvlan.vid=100, send nni1
Repl-member2 = edit cvlan.vid=200, send nni2

member1_nni:packet = eth, cvlan, rtag
member1_nni:match = cvlan vid=100
member1_nni:actions = readseq rtag, Elim Elim-compound
```

```
member2_nni:packet = eth, cvlan, rtag
member2_nni:match = cvlan vid=200
member2_nni:actions = readseq rtag, Elim Elim-compound

Elim-compound = del rtag, del cvlan, send uni
```

Steps to validate Objective 2 from the shell:

Capture from a neutral vantage, not an edge: an inner bridge does not run DNT, so it does not contend with DNT's socket. On an inner Path 1 bridge, for example Bridge 1A, capture the interface and filter for the FRER headers. Tshark is command-line Wireshark.

- On Bridge1A, or any bridge that is not an edge bridge, run:
  - `tshark -O 'ieee8021cb' -i eth0 | grep -e 802 -e "10.0.0.1"`
- **Generate traffic from Host A by running:**
  - `ping -c 4 10.0.0.2`
- Stop the capture running on Bridge1A or your set bridge
  - `ctrl-c`

Validating the Application Connectivity:

- In this screenshot you can see we are pinging from talker with 10 counts to listener

```
rtt min/avg/max/mdev = 0.876/0.989/1.137/0.078 ms
root@h1:/tmp/munet/h1# ping -c 10 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.988 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=1.08 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.848 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.930 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.986 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.992 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.778 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.903 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.906 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=1.02 ms

--- 10.0.0.2 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9039ms
rtt min/avg/max/mdev = 0.778/0.943/1.081/0.084 ms
root@h1:/tmp/munet/h1#
```

Life of a Packet

Packet capture on Path 1 taken at bridge1A on eth1-net1B

- Here is a packet capture of Path1 taken from Bridge1A eth1. In this capture, you can see that packets are encapsulated with a VLAN ID as well as an R-Tag demonstrating DNT's desired encapsulation sequence

```
No.    Time    Source    Destination    Protocol    Length    Info
Frame 1: Packet, 108 bytes on wire (864 bits), 108 bytes captured (864 bits)
    Encapsulation type: Ethernet (1)
    Arrival Time: Jul 17, 2026 21:43:29.105260000 EDT
    UTC Arrival Time: Jul 18, 2026 01:43:29.105260000 UTC
    Epoch Arrival Time: 1784339009.105260000
    [Time shift for this packet: 0.000000000 seconds]
    [Time since reference or first frame: 0.000000000 seconds]
    Frame Number: 1
    Frame Length: 108 bytes (864 bits)
    Capture Length: 108 bytes (864 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:vlan:ethertype:rtag:ethertype:ip:icmp:data]
    Character encoding: ASCII (0)
    [Coloring Rule Name: ICMP]
    [Coloring Rule String: icmp || icmpv6]
Ethernet II, Src: b6:54:da:61:25:fe (b6:54:da:61:25:fe), Dst: 9a:35:dc:b9:c7:5b (9a:35:dc:b9:c7:5b)
  > Destination: 9a:35:dc:b9:c7:5b (9a:35:dc:b9:c7:5b)
  > Source: b6:54:da:61:25:fe (b6:54:da:61:25:fe)
    Type: 802.1Q Virtual LAN (0x8100)
    [Stream index: 0]
802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 100
    000. .... .... .... = Priority: Best Effort (default) (0)
    ...0 .... .... .... = DEI: Ineligible
    .... 0000 0110 0100 = ID: 100
    Type: 802.1CB Frame Replication and Elimination for Reliability (0xf1c1)
802.1cb R-TAG
    <reserved>: 0x0000
    Sequence number: 108
    Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 84
    Identification: 0x05e0 (1504)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: ICMP (1)
    Header Checksum: 0x20c7 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.0.0.1
    Destination Address: 10.0.0.2
    [Stream index: 0]
Internet Control Message Protocol
    Type: Echo (ping) request (8)
    Code: 0
    Checksum: 0x861a [correct]
    [Checksum Status: Good]
    Identifier (BE): 51 (0x0033)
    Identifier (LE): 13056 (0x3300)
```

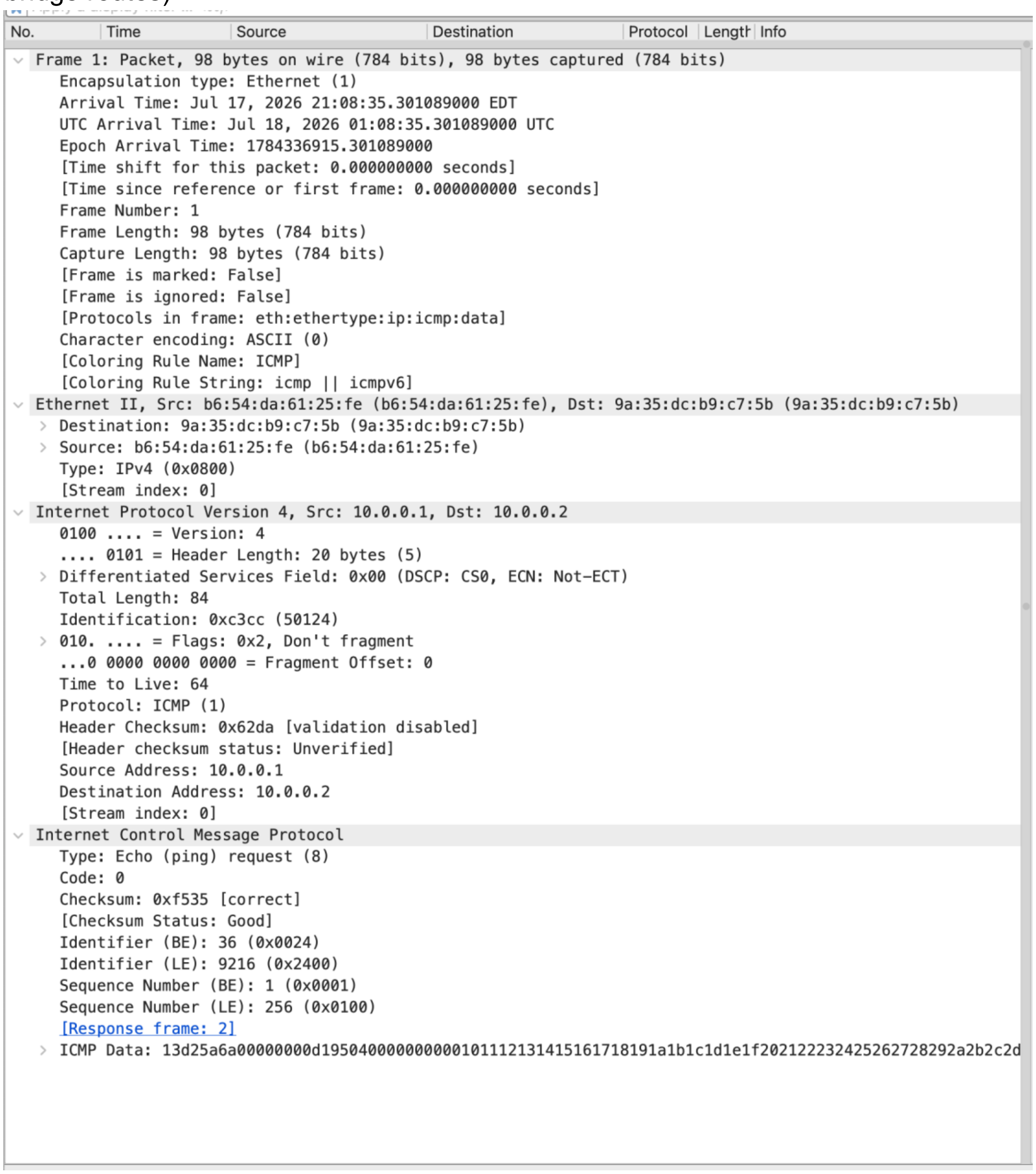
Packet capture on Path 2, taken at bridge2A on eth1-net2B

- Here is a packet capture of Path2 taken from Bridge2A eth1. This capture is identical to the Path1 capture except that the VLAN ID is 200 rather than 100. The R-Tag sequence numbers match Path1 packet for packet, demonstrating that DNT replication


```
Frame 1: Packet, 108 bytes on wire (864 bits), 108 bytes captured (864 bits)
    Encapsulation type: Ethernet (1)
    Arrival Time: Jul 17, 2026 21:43:29.105437000 EDT
    UTC Arrival Time: Jul 18, 2026 01:43:29.105437000 UTC
    Epoch Arrival Time: 1784339009.105437000
    [Time shift for this packet: 0.000000000 seconds]
    [Time since reference or first frame: 0.000000000 seconds]
    Frame Number: 1
    Frame Length: 108 bytes (864 bits)
    Capture Length: 108 bytes (864 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:vlan:ethertype:rtag:ethertype:ip:icmp:data]
    Character encoding: ASCII (0)
    [Coloring Rule Name: ICMP]
    [Coloring Rule String: icmp || icmpv6]
Ethernet II, Src: b6:54:da:61:25:fe (b6:54:da:61:25:fe), Dst: 9a:35:dc:b9:c7:5b (9a:35:dc:b9:c7:5b)
    Destination: 9a:35:dc:b9:c7:5b (9a:35:dc:b9:c7:5b)
    Source: b6:54:da:61:25:fe (b6:54:da:61:25:fe)
    Type: 802.1Q Virtual LAN (0x8100)
    [Stream index: 0]
802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 200
    000. .... .... .... = Priority: Best Effort (default) (0)
    ...0 .... .... .... = DEI: Ineligible
    .... 0000 1100 1000 = ID: 200
    Type: 802.1CB Frame Replication and Elimination for Reliability (0xf1c1)
802.1cb R-TAG
    <reserved>: 0x0000
    Sequence number: 108
    Type: IPv4 (0x0800)
Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
    Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 84
    Identification: 0x05e0 (1504)
    010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: ICMP (1)
    Header Checksum: 0x20c7 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.0.0.1
    Destination Address: 10.0.0.2
    [Stream index: 0]
Internet Control Message Protocol
    Type: Echo (ping) request (8)
    Code: 0
    Checksum: 0x861a [correct]
    [Checksum Status: Good]
    Identifier (BE): 51 (0x0033)
    Identifier (LE): 13056 (0x3300)
```

Packet capture on the listener, taken at h2 on eth0, link-h2

- Here is a packet capture from Listener, as you can see listener only receives 1 packet instead of 2 (note these are separate packets from the ones above from the separate bridge routes)



Mutest:

```
INFO: NUMBER     STAT   TARGET   TIME  DESCRIPTION
INFO: ----------------------------------------------------------------------
INFO:
INFO: 1. Objective 2: FRER (802.1CB) over transparent Layer 2 bridges.
```

```
INFO:
INFO: 1.1. The DNT replication engine is running on both edge nodes
INFO: 1.1.1      PASS  bridgeA   0.01  At least one dnt process is running with
dnt.ini
INFO: 1.1.2      PASS  bridgeB  0.008  At least one dnt process is running with
dnt.ini
INFO:
INFO: 1.2. The four relay nodes are plain Linux bridges, nothing more
INFO: 1.2.1      PASS bridge1A  0.006  br0 exists and is a real bridge device
INFO: 1.2.2      PASS bridge1A  0.005  br0 is administratively up
INFO: 1.2.3      PASS bridge1A  0.004  eth0 is enslaved to br0
INFO: 1.2.4      PASS bridge1A  0.004  eth1 is enslaved to br0
INFO: 1.2.5      PASS bridge1B  0.004  br0 exists and is a real bridge device
INFO: 1.2.6      PASS bridge1B  0.004  br0 is administratively up
INFO: 1.2.7      PASS bridge1B  0.005  eth0 is enslaved to br0
INFO: 1.2.8      PASS bridge1B  0.004  eth1 is enslaved to br0
INFO: 1.2.9      PASS bridge2A  0.005  br0 exists and is a real bridge device
INFO: 1.2.10     PASS bridge2A  0.004  br0 is administratively up
INFO: 1.2.11     PASS bridge2A  0.004  eth0 is enslaved to br0
INFO: 1.2.12     PASS bridge2A  0.004  eth1 is enslaved to br0
INFO: 1.2.13     PASS bridge2B  0.004  br0 exists and is a real bridge device
INFO: 1.2.14     PASS bridge2B  0.005  br0 is administratively up
INFO: 1.2.15     PASS bridge2B  0.004  eth0 is enslaved to br0
INFO: 1.2.16     PASS bridge2B  0.004  eth1 is enslaved to br0
INFO:
INFO: 1.3. The core has no IP addresses and no routing daemon anywhere
INFO: 1.3.1      PASS bridge1A  0.006  Neither relay port carries an IPv4 address
INFO: 1.3.2      PASS bridge1B  0.005  Neither relay port carries an IPv4 address
INFO: 1.3.3      PASS bridge2A  0.005  Neither relay port carries an IPv4 address
INFO: 1.3.4      PASS bridge2B  0.005  Neither relay port carries an IPv4 address
INFO: 1.3.5      PASS bridge1A  0.006  No zebra, ospfd or frr process is running
INFO: 1.3.6      PASS       h1  0.004  h1 and h2 share one subnet, with no router
between them
INFO:
INFO: 1.4. Host ARP entries are pinned so replicated ARP stays out of the captures
INFO: 1.4.1      PASS       h1    1.1  Warm-up ping from h1 to h2 got 2 of 2 replies
INFO: 1.4.2      PASS       h1    0.0  The warm-up ping had no loss and no duplicate
replies
INFO:
INFO: 1.5. Captures are running on both paths and h1 sends 10 pings
INFO: 1.5.1      PASS bridge1A    0.5  tcpdump listening on eth1 for the path1
capture
INFO: 1.5.2      PASS bridge2A    0.5  tcpdump listening on eth1 for the path2
capture
INFO: 1.5.3      PASS       h2    0.5  tcpdump listening on eth0 for the listener
capture
```

```
INFO: 1.5.4      PASS       h1    9.1  h1 received all 10 ICMP replies
INFO: 1.5.5      PASS       h1    0.0  h1 reported 0% packet loss
INFO: 1.5.6      PASS       h1    0.0  h1 had no loss and saw no duplicates, so
elimination worked
INFO: 1.5.7      PASS bridge1A    2.5  The path1 capture flushed to disk and stopped
INFO: 1.5.8      PASS bridge2A    0.5  The path2 capture flushed to disk and stopped
INFO: 1.5.9      PASS       h2    0.5  The listener capture flushed to disk and
stopped
INFO:
INFO: 1.6. What is actually on the wire, surveyed before anything is asserted
INFO: 1.6.1      PASS bridge1A   0.01  Count every frame captured on Path 1
INFO: 1.6.2      PASS bridge2A   0.01  Count every frame captured on Path 2
INFO: 1.6.3      PASS bridge1A    0.0  Path 1 carried 140 frames of any kind
INFO: 1.6.4      PASS bridge2A    0.0  Path 2 carried 140 frames of any kind
INFO: 1.6.5      PASS bridge1A  0.009  Read the first Path 1 frame off the capture
INFO: 1.6.6      PASS bridge2A  0.008  Read the first Path 2 frame off the capture
INFO: 1.6.7      PASS bridge1A  0.008  Count Path 1 frames with ethertype 0x8100 at
byte offset 12
INFO: 1.6.8      PASS bridge1A  0.007  Count Path 1 frames with ethertype 0xf1c1 at
byte offset 12
INFO: 1.6.9      PASS bridge1A  0.008  Count Path 1 frames with ethertype 0xf1c1 at
byte offset 16
INFO: 1.6.10     PASS bridge1A  0.007  Count Path 1 frames with ethertype 0x88a8 at
byte offset 12
INFO:
INFO: 1.7. REPLICATION. Both paths carry their own copy of every frame
INFO: 1.7.1      PASS bridge1A  0.007  Count Path 1 frames on VLAN 100
INFO: 1.7.2      PASS bridge2A  0.007  Count Path 2 frames on VLAN 200
INFO: 1.7.3      PASS bridge1A    0.0  Path 1 carried 140 frames tagged VLAN 100
INFO: 1.7.4      PASS bridge2A    0.0  Path 2 carried 140 frames tagged VLAN 200
INFO: 1.7.5      PASS             0.0  Both paths carried the same number of copies,
140 and 140
INFO:
INFO: 1.8. SEPARATION. Each path carries only its own VLAN, never the other's
INFO: 1.8.1      PASS bridge1A  0.008  Count VLAN 200 frames wrongly on Path 1
INFO: 1.8.2      PASS bridge2A  0.007  Count VLAN 100 frames wrongly on Path 2
INFO: 1.8.3      PASS bridge1A    0.0  Path 1 was busy and carried no VLAN 200 frames
at all (0)
INFO: 1.8.4      PASS bridge2A    0.0  Path 2 was busy and carried no VLAN 100 frames
at all (0)
INFO:
INFO: 1.9. R-TAG. Every replicated frame carries an 802.1CB sequence tag
INFO: 1.9.1      PASS bridge1A  0.008  Count Path 1 frames carrying an 802.1CB R-TAG
INFO: 1.9.2      PASS bridge2A  0.007  Count Path 2 frames carrying an 802.1CB R-TAG
INFO: 1.9.3      PASS bridge1A    0.0  Path 1 carried 140 R-TAGged frames
INFO: 1.9.4      PASS bridge2A    0.0  Path 2 carried 140 R-TAGged frames
```

```
INFO: 1.9.5      PASS bridge1A    0.0  Every one of Path 1's frames was R-TAGged, 140
of 140
INFO: 1.9.6      PASS bridge2A    0.0  Every one of Path 2's frames was R-TAGged, 140
of 140
INFO:
INFO: 1.10. ELIMINATION. The listener receives one copy of each frame, not two
INFO: 1.10.1     PASS       h2  0.008  Count ICMP echo requests arriving at the
listener
INFO: 1.10.2     PASS       h2  0.007  Count ICMP echo replies sent back by the
listener
INFO: 1.10.3     PASS       h2    0.0  h2 received 10 requests, one per ping (10)
INFO: 1.10.4     PASS       h2    0.0  h2 sent 10 replies, one per request (10)
INFO: 1.10.5     PASS       h2    0.0  280 frames crossed the core but only 10
reached h2
INFO: 1.10.6     PASS       h2  0.007  Count still-tagged frames arriving at the
listener
INFO: 1.10.7     PASS       h2    0.0  h2 sees plain untagged frames, 0 still tagged
INFO:
INFO: 1.11. PATH 1 DOWN. Traffic keeps flowing on Path 2 alone
INFO: 1.11.1     PASS bridge1A    0.5  tcpdump listening on eth1 for the p1down-path1
capture
INFO: 1.11.2     PASS bridge2A    0.5  tcpdump listening on eth1 for the p1down-path2
capture
INFO: 1.11.3     PASS       h2    0.5  tcpdump listening on eth0 for the p1down-
listener capture
INFO: 1.11.4     PASS bridge1A   11.6  The p1down-path1 capture flushed to disk and
stopped
INFO: 1.11.5     PASS bridge2A    0.5  The p1down-path2 capture flushed to disk and
stopped
INFO: 1.11.6     PASS       h2    0.5  The p1down-listener capture flushed to disk
and stopped
INFO: 1.11.7     PASS bridge1A   0.01  Count Path 1 frames sent by h1, with Path 1
down
INFO: 1.11.8     PASS bridge1A   0.01  Count Path 1 frames sent back by h2, with Path
1 down
INFO: 1.11.9     PASS bridge2A  0.008  Count Path 2 frames sent by h1, with Path 1
down
INFO: 1.11.10    PASS bridge2A  0.008  Count Path 2 frames sent back by h2, with Path
1 down
INFO: 1.11.11    PASS       h2  0.008  Count requests delivered to the listener, with
Path 1 down
INFO: 1.11.12    PASS bridge1A    0.0  Path 1 sent nothing from h1, 0 frames
INFO: 1.11.13    PASS bridge2A    0.0  Path 2 kept sending from h1, 70 frames
INFO: 1.11.14    PASS bridge1A    0.0  h2's replies still came back down Path 1, 70
INFO: 1.11.15    PASS bridge2A    0.0  h2's replies also came down Path 2, 70
INFO: 1.11.16    PASS       h2    0.0  h2 still received all 10 requests (10)
INFO: 1.11.17    PASS       h1    0.0  0% loss with Path 1 down
```

```
INFO: 1.11.18    PASS       h1    0.0  No loss and no duplicates with Path 1 down
INFO:
INFO: 1.12. PATH 2 DOWN. Traffic keeps flowing on Path 1 alone
INFO: 1.12.1     PASS  bridgeA  0.009  The Path 1 port came back up
INFO: 1.12.2     PASS bridge1A    0.5  tcpdump listening on eth1 for the p2down-path1
capture
INFO: 1.12.3     PASS bridge2A    0.5  tcpdump listening on eth1 for the p2down-path2
capture
INFO: 1.12.4     PASS       h2    0.5  tcpdump listening on eth0 for the p2down-
listener capture
INFO: 1.12.5     PASS bridge1A   11.6  The p2down-path1 capture flushed to disk and
stopped
INFO: 1.12.6     PASS bridge2A    0.5  The p2down-path2 capture flushed to disk and
stopped
INFO: 1.12.7     PASS       h2    0.5  The p2down-listener capture flushed to disk
and stopped
INFO: 1.12.8     PASS bridge1A   0.01  Count Path 1 frames sent by h1, with Path 2
down
INFO: 1.12.9     PASS bridge1A  0.009  Count Path 1 frames sent back by h2, with Path
2 down
INFO: 1.12.10    PASS bridge2A  0.009  Count Path 2 frames sent by h1, with Path 2
down
INFO: 1.12.11    PASS bridge2A  0.010  Count Path 2 frames sent back by h2, with Path
2 down
INFO: 1.12.12    PASS       h2  0.008  Count requests delivered to the listener, with
Path 2 down
INFO: 1.12.13    PASS bridge2A    0.0  Path 2 sent nothing from h1, 0 frames
INFO: 1.12.14    PASS bridge1A    0.0  Path 1 kept sending from h1, 70 frames
INFO: 1.12.15    PASS bridge2A    0.0  h2's replies still came back down Path 2, 70
INFO: 1.12.16    PASS bridge1A    0.0  h2's replies also came down Path 1, 70
INFO: 1.12.17    PASS       h2    0.0  h2 still received all 10 requests (10)
INFO: 1.12.18    PASS       h1    0.0  0% loss with Path 2 down
INFO: 1.12.19    PASS       h1    0.0  No loss and no duplicates with Path 2 down
INFO:
INFO: 1.13. BOTH PATHS BACK. Replication resumes on both at once
INFO: 1.13.1     PASS  bridgeA  0.009  The Path 2 port came back up
INFO: 1.13.2     PASS bridge1A    0.5  tcpdump listening on eth1 for the bothup-path1
capture
INFO: 1.13.3     PASS bridge2A    0.5  tcpdump listening on eth1 for the bothup-path2
capture
INFO: 1.13.4     PASS       h2    0.5  tcpdump listening on eth0 for the bothup-
listener capture
INFO: 1.13.5     PASS bridge1A   11.6  The bothup-path1 capture flushed to disk and
stopped
INFO: 1.13.6     PASS bridge2A    0.5  The bothup-path2 capture flushed to disk and
stopped
```

```
INFO: 1.13.7     PASS       h2    0.5  The bothup-listener capture flushed to disk
and stopped
INFO: 1.13.8     PASS bridge1A   0.01  Count Path 1 frames sent by h1, with both
paths up
INFO: 1.13.9     PASS bridge1A  0.010  Count Path 1 frames sent back by h2, with both
paths up
INFO: 1.13.10    PASS bridge2A  0.009  Count Path 2 frames sent by h1, with both
paths up
INFO: 1.13.11    PASS bridge2A  0.008  Count Path 2 frames sent back by h2, with both
paths up
INFO: 1.13.12    PASS       h2  0.008  Count requests delivered to the listener, with
both paths up
INFO: 1.13.13    PASS bridge1A    0.0  Path 1 is sending from h1 again, 70 frames
INFO: 1.13.14    PASS bridge2A    0.0  Path 2 is sending from h1 again, 70 frames
INFO: 1.13.15    PASS             0.0  Both paths are carrying equally again, 70 and
70
INFO: 1.13.16    PASS             0.0  Both paths carry h2's replies too, 70 and 70
INFO: 1.13.17    PASS       h2    0.0  h2 received all 10 requests (10)
INFO: 1.13.18    PASS       h1    0.0  0% loss with both paths up
INFO: 1.13.19    PASS       h1    0.0  No loss and no duplicates with both paths up
INFO:
INFO: run stats: 123 steps, 123 pass, 0 fail, 0 abort, 61.51s elapsed
INFO: ----------------------------------------------------------------------
INFO:  PASS  1:mutest_obj2_frer
INFO: ----------------------------------------------------------------------
INFO: END RUN: 1 test scripts, 1 passed, 0 failed
```

6. OBJECTIVE 3

Set up topology that supports PREOF using static routers, with no FRR

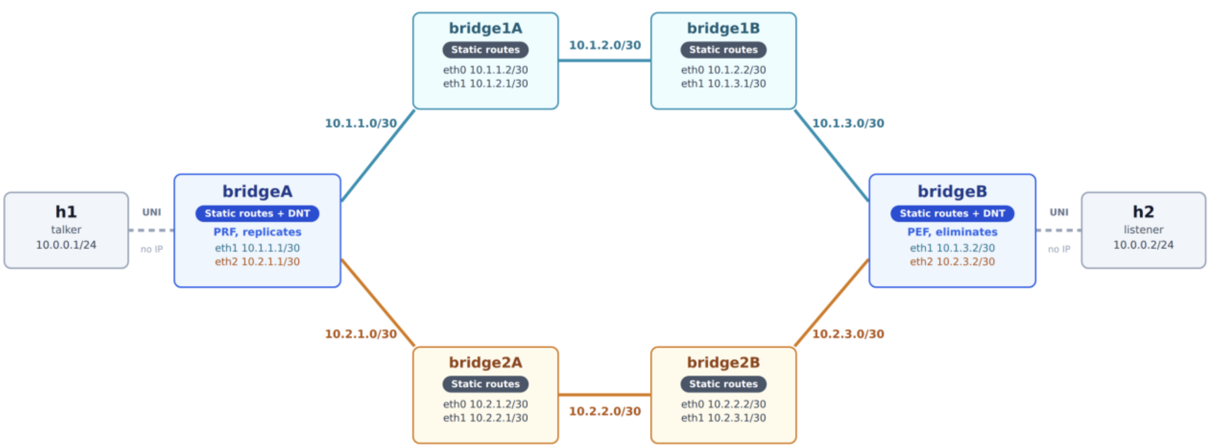
Layer 3 redundancy, PREOF, over static routes only, with no routing protocol anywhere. Ping ran 5 of 5 at 0% loss, and the Path 1 capture decoded the full DetNet stack: UDP 6635, MPLS label 100, control word sequence, inner VLAN 99, original ICMP. The core forwarded plain IP and never saw MPLS as a control plane, so Objective 4 can swap in OSPF without touching PREOF. PREOF (Packet Replication, Elimination, and Ordering Functions) is the DetNet (Deterministic Networking, an IETF standard) Layer 3 reliability mechanism: it sends a copy of every packet down two disjoint paths and drops the duplicate at the far end, so a fault on one path causes no loss. It is the Layer 3 equivalent of Objective 2's FRER. DNT, the userspace toolkit from Objective 1, runs the two active functions, PRF (Packet Replication Function) and PEF (Packet Elimination Function), keyed by a per-packet sequence number. On this Layer 3 topology DNT carries each copy in an MPLS-over-UDP pseudowire: it wraps the original frame in an MPLS label plus a DetNet control word that holds the sequence number, then tunnels that inside a normal UDP/IP packet sent between the two edge routers on UDP port 6635.

Steps to achieve Objective 3

Step 1. Identify Topology

PREOF needs two node-disjoint paths and one edge router at each end doing the replicate and eliminate work. Building the core with static routes and no routing protocol isolates PREOF, so it can be validated before FRR and OSPF are added in Objective 4. The static routes also make the key architectural point visible: the core routers only need plain IP reachability to carry DNT's UDP tunnel, they never run MPLS as a control plane.
Eight nodes and eight links, the same shape and node names as the Objective 2 FRER topology, with the four inner bridges now working as static routers instead of Linux bridges:

- Host A and Host B: the end hosts (talker and listener), both on subnet 10.0.0.0/24
- Edge Router 1 and Edge Router 2: the edges, now Layer 3 and running DNT PREOF. Edge Router 1 replicates (PRF) at ingress, Edge Router 2 eliminates (PEF) at egress.
- Path 1 is Router 1A then Router 1B. Path 2 is Router 2A then Router 2B. These four inner nodes are plain static routers that forward by static route only.
- Disjoint paths: the two paths share no node, so a fault on one cannot affect the other. This is the property PREOF depends on.



Step 2. Prepare the environment:

No new tools are needed. The Objective 1 VM already has DNT built and installed, and munet is installed from Objective 2. The preparation is only to create a working directory for this scenario and place its three config files inside it.

- First, create a working directory on the VM to hold everything for Objective 3, for example ~/lab/preof-topo. The paths inside munet.yaml point at this directory, so the whole scenario stays self-contained.

- Then create three files inside it:
  - munet.yaml: the topology, all eight nodes with their links, addresses, static routes, and the DNT launch on each edge.
  - routerA.ini: the DNT PREOF config for Edge Router 1.
  - routerB.ini: the DNT PREOF config for Edge Router 2.

Populate each file with the contents given in the config listings; they are not repeated here. The directory ends up as:

```
▪ ~/lab/preof-topo/munet.yaml
  routerA.ini
  routerB.ini
```

The whole topology is one munet YAML file. It declares eight point-to-point networks and the eight nodes, and each node's cmd runs at bring-up: the hosts set their address and disable offload, the four intermediates enable IP forwarding and add one static route toward the far edge, and the two edges enable forwarding, add both static routes, and launch DNT. This is the topology code:

Munet Configuration:

```
topology:
  networks:
  - name: link-h1
  - name: net1A
  - name: net1B
  - name: net1C
  - name: net2A
  - name: net2B
  - name: net2C
  - name: link-h2

  nodes:
  - name: h1
    cmd: |
      ip addr add 10.0.0.1/24 dev eth0
      ip link set eth0 up
      ethtool -K eth0 rxvlan off txvlan off tx off rx off
      bash
    connections:
    - to: link-h1
```

```yaml
- name: routerA
  cmd: |
    sysctl -w net.ipv4.ip_forward=1
    ip link set eth0 up
    ip link set eth1 mtu 1600 up
    ip link set eth2 mtu 1600 up
    ip addr add 10.1.1.1/30 dev eth1
    ip addr add 10.2.1.1/30 dev eth2
    ip route add 10.1.3.2/32 via 10.1.1.2 dev eth1
    ip route add 10.2.3.2/32 via 10.2.1.2 dev eth2
    /usr/local/bin/dnt /home/francis/lab/preof-topo/routerA.ini
  connections:
  - to: link-h1
  - to: net1A
  - to: net2A

- name: router1A
  cmd: |
    sysctl -w net.ipv4.ip_forward=1
    ip link set eth0 mtu 1600 up
    ip link set eth1 mtu 1600 up
    ip addr add 10.1.1.2/30 dev eth0
    ip addr add 10.1.2.1/30 dev eth1
    ip route add 10.1.3.2/32 via 10.1.2.2 dev eth1
  connections:
  - to: net1A
  - to: net1B

- name: router1B
  cmd: |
    sysctl -w net.ipv4.ip_forward=1
    ip link set eth0 mtu 1600 up
    ip link set eth1 mtu 1600 up
    ip addr add 10.1.2.2/30 dev eth0
    ip addr add 10.1.3.1/30 dev eth1
    ip route add 10.1.1.1/32 via 10.1.2.1 dev eth0
  connections:
  - to: net1B
  - to: net1C

- name: router2A
  cmd: |
    sysctl -w net.ipv4.ip_forward=1
    ip link set eth0 mtu 1600 up
    ip link set eth1 mtu 1600 up
    ip addr add 10.2.1.2/30 dev eth0
```

```
      ip addr add 10.2.2.1/30 dev eth1
      ip route add 10.2.3.2/32 via 10.2.2.2 dev eth1
    connections:
    - to: net2A
    - to: net2B

  - name: router2B
    cmd: |
      sysctl -w net.ipv4.ip_forward=1
      ip link set eth0 mtu 1600 up
      ip link set eth1 mtu 1600 up
      ip addr add 10.2.2.2/30 dev eth0
      ip addr add 10.2.3.1/30 dev eth1
      ip route add 10.2.1.1/32 via 10.2.2.1 dev eth0
    connections:
    - to: net2B
    - to: net2C

  - name: routerB
    cmd: |
      sysctl -w net.ipv4.ip_forward=1
      ip link set eth0 up
      ip link set eth1 mtu 1600 up
      ip link set eth2 mtu 1600 up
      ip addr add 10.1.3.2/30 dev eth1
      ip addr add 10.2.3.2/30 dev eth2
      ip route add 10.1.1.1/32 via 10.1.3.1 dev eth1
      ip route add 10.2.1.1/32 via 10.2.3.1 dev eth2
      /usr/local/bin/dnt /home/francis/lab/preof-topo/routerB.ini
    connections:
    - to: link-h2
    - to: net1C
    - to: net2C

  - name: h2
    cmd: |
      ip addr add 10.0.0.2/24 dev eth0
      ip link set eth0 up
      ethtool -K eth0 rxvlan off txvlan off tx off rx off
      bash
    connections:
    - to: link-h2
```

DNT RouterA:

```
[interfaces]
uni = eth iface=eth0
```

```
uni:streams = prio-compound compound

nni1_in = udp-in iface=eth1 ipv=4
nni1_in:streams = member1 prio-members
nni2_in = udp-in iface=eth2 ipv=4
nni2_in:streams = member2 prio-members

nni1_out = udp-out iface=eth1 dstip=10.1.3.2
nni2_out = udp-out iface=eth2 dstip=10.2.3.2

[objects]
prf = Replicate
gen = SeqGen InitSeqStart=0
pef = SeqRcvy

genprio = SeqGen InitSeqStart=0
prfprio = Replicate
pefprio = SeqRcvy

[streams]
compound:packet = eth, cvlan
compound:match = cvlan vid=0
compound:actions = gen, edit cvlan.vid=99, before eth add mpls, after mpls add dcw,
writeseq dcw, prf prf-member1 prf-member2

prf-member1 = edit mpls.label=100 mpls.bos=1, send nni1_out
prf-member2 = edit mpls.label=200 mpls.bos=1, send nni2_out

member1:packet = mpls, dcw, eth, cvlan
member1:match = mpls label=100
member1:actions = readseq dcw, pef pef-compound

member2:packet = mpls, dcw, eth, cvlan
member2:match = mpls label=200
member2:actions = readseq dcw, pef pef-compound

pef-compound = del dcw, del mpls, del cvlan, send uni

prio-compound:packet = eth, cvlan
prio-compound:match = cvlan vid=0 pcp=6
prio-compound:actions = genprio, edit cvlan.vid=200, before eth add mpls, after mpls
add dcw, writeseq dcw, prfprio prfprio-member1 prfprio-member2

prfprio-member1 = edit mpls.label=500 mpls.bos=1, send nni1_out
prfprio-member2 = edit mpls.label=500 mpls.bos=1, send nni2_out

prio-members:packet = mpls, dcw, eth, cvlan
```

```
prio-members:match = mpls label=500
prio-members:actions = readseq dcw, pefprio pefprio-compound

pefprio-compound = del dcw, del mpls, edit cvlan.vid=0, send uni
```

DNT RouterB:

```
[interfaces]
uni = eth iface=eth0
uni:streams = prio-compound compound

nni1_in = udp-in iface=eth1 ipv=4
nni1_in:streams = member1 prio-members
nni2_in = udp-in iface=eth2 ipv=4
nni2_in:streams = member2 prio-members

nni1_out = udp-out iface=eth1 dstip=10.1.1.1
nni2_out = udp-out iface=eth2 dstip=10.2.1.1

[objects]
prf = Replicate
gen = SeqGen InitSeqStart=0
pef = SeqRcvy

genprio = SeqGen InitSeqStart=0
prfprio = Replicate
pefprio = SeqRcvy

[streams]
compound:packet = eth, cvlan
compound:match = cvlan vid=0
compound:actions = gen, edit cvlan.vid=99, before eth add mpls, after mpls add dcw,
writeseq dcw, prf prf-member1 prf-member2

prf-member1 = edit mpls.label=100 mpls.bos=1, send nni1_out
prf-member2 = edit mpls.label=200 mpls.bos=1, send nni2_out

member1:packet = mpls, dcw, eth, cvlan
member1:match = mpls label=100
member1:actions = readseq dcw, pef pef-compound

member2:packet = mpls, dcw, eth, cvlan
member2:match = mpls label=200
member2:actions = readseq dcw, pef pef-compound

pef-compound = del dcw, del mpls, del cvlan, send uni
```

```
prio-compound:packet = eth, cvlan
prio-compound:match = cvlan vid=0 pcp=6
prio-compound:actions = genprio, edit cvlan.vid=200, before eth add mpls, after mpls
add dcw, writeseq dcw, prfprio prfprio-member1 prfprio-member2

prfprio-member1 = edit mpls.label=500 mpls.bos=1, send nni1_out
prfprio-member2 = edit mpls.label=500 mpls.bos=1, send nni2_out

prio-members:packet = mpls, dcw, eth, cvlan
prio-members:match = mpls label=500
prio-members:actions = readseq dcw, pefprio pefprio-compound

pefprio-compound = del dcw, del mpls, edit cvlan.vid=0, send uni
```

Bringing up munet already launched DNT on both edges from the edge command above. This is the config it loaded (routerA.ini). The [interfaces] section names the UNI port and the two UDP tunnel outputs, one per path, each with the far edge's IP. The [objects] section defines the replicate, sequence-generate, and recovery functions. The [streams] section is the PREOF logic: on ingress DNT adds a sequence number, wraps the frame in an MPLS label plus a DetNet control word, writes the sequence into the control word, and replicates onto Path 1 (label 100) and Path 2 (label 200); on return it matches the label, reads the sequence back, drops duplicates, strips the encapsulation, and delivers one clean copy:

Steps to validate from the shell:

- Confirm both paths route edge to edge at plain IP across both intermediate hops, independent of DNT. Run from Edge Router 1. The route lookup should resolve via the first intermediate (10.1.1.2 on eth1), and both pings should reach Edge Router 2 across their two hops:

    - ```
      ip route get 10.1.3.2
      ping -c2 10.1.3.2
      ping -c2 10.2.3.2
      ```

- Stage 2, PREOF on the wire. Capture from a neutral vantage, an intermediate router that does not run DNT so it does not contend for the interface. On Router 1A, capture Path 1 filtered to DNT's tunnel port:

    - ```
      tcpdump -ni eth1 -w preof_path1.pcap udp port 6635
      ```

    - On Router 2A, capture Path 2

        - ```
          tcpdump -ni eth1 -w preof_path2.pcap udp port 6635
          ```
          ```

- Stage 3, confirm no dynamic routing, the defining constraint of the objective. On every node, no routing daemon runs and the tables are static only. The process check should return nothing, and each route table should show only the connected /30 links plus the static /32 routes:

  - ```
    ps -ef | grep -E 'zebra|ospfd|frr'
    ip route show
    ```

Validating the Application Connectivity:

- In this screenshot you can see we are pinging from talker with 10 counts to listener

```
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.384 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.421 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.501 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.510 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.390 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.317 ms
64 bytes from 10.0.0.2: icmp_seq=7 ttl=64 time=0.303 ms
64 bytes from 10.0.0.2: icmp_seq=8 ttl=64 time=0.427 ms
64 bytes from 10.0.0.2: icmp_seq=9 ttl=64 time=0.391 ms
64 bytes from 10.0.0.2: icmp_seq=10 ttl=64 time=0.401 ms

--- 10.0.0.2 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9212ms
rtt min/avg/max/mdev = 0.303/0.404/0.510/0.063 ms
root@h1:/tmp/munet/h1#
```

Life of a Packet:

Packet capture on router1A eth1

- Here is a packet captured on router1A, the packet is being sent from talker to listener. Packets on each route is encapsulated with their configured member information. In the screenshot below the packet’s MPLS header ID(100), nd the VLAN ID(99) match the member configuration in the DNT setup.
```

```
No.   Time   Source   Destination   Protocol   Length   Info
    [Time shift for this packet: 0.000000000 seconds]
    [Time since reference or first frame: 0.000000000 seconds]
    Frame Number: 1
    Frame Length: 152 bytes (1216 bits)
    Capture Length: 152 bytes (1216 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:udp:mpls:pwethcw:eth:ethertype:vlan:ethertype:ip:icmp:data]
    Character encoding: ASCII (0)
    [Coloring Rule Name: ICMP]
    [Coloring Rule String: icmp || icmpv6]
v Ethernet II, Src: 16:8a:eb:2e:60:e4 (16:8a:eb:2e:60:e4), Dst: 2e:90:17:f1:e3:9a (2e:90:17:f1:e3:9a)
  > Destination: 2e:90:17:f1:e3:9a (2e:90:17:f1:e3:9a)
  > Source: 16:8a:eb:2e:60:e4 (16:8a:eb:2e:60:e4)
    Type: IPv4 (0x0800)
    [Stream index: 0]
v Internet Protocol Version 4, Src: 10.1.1.1, Dst: 10.1.3.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 138
    Identification: 0x0000 (0)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 63
    Protocol: UDP (17)
    Header Checksum: 0x235f [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.1.1.1
    Destination Address: 10.1.3.2
    [Stream index: 0]
> User Datagram Protocol, Src Port: 46005, Dst Port: 6635
v MultiProtocol Label Switching Header, Label: 100, Exp: 0, S: 1, TTL: 0
    0000 0000 0000 0110 0100 .... .... .... = MPLS Label: 100 (0x00064)
    .... .... .... .... .... 000. .... .... = MPLS Experimental Bits: 0
    .... .... .... .... .... ...1 .... .... = MPLS Bottom Of Label Stack: 1
    .... .... .... .... .... .... 0000 0000 = MPLS TTL: 0
v PW Ethernet Control Word
    Sequence Number: 2
v Ethernet II, Src: be:16:8a:20:2e:42 (be:16:8a:20:2e:42), Dst: ae:cd:e7:12:64:04 (ae:cd:e7:12:64:04)
  > Destination: ae:cd:e7:12:64:04 (ae:cd:e7:12:64:04)
  > Source: be:16:8a:20:2e:42 (be:16:8a:20:2e:42)
    Type: 802.1Q Virtual LAN (0x8100)
    [Stream index: 1]
v 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 99
    000. .... .... .... = Priority: Best Effort (default) (0)
    ...0 .... .... .... = DEI: Ineligible
    .... 0000 0110 0011 = ID: 99
    Type: IPv4 (0x0800)
v Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
    0100 .... = Version: 4
```

Packet capture on listener

- Here is a packet capture from Listener, as you can see listener only receives 1 packet instead of 2 stripped from the member configuration (VLAN ID, MPLS, sequence ID) returning into a regular ICMP packet

```
No.          Time            Source                    Destination               Protocol  Length  Info
∨ Frame 1: Packet, 98 bytes on wire (784 bits), 98 bytes captured (784 bits)
    Encapsulation type: Ethernet (1)
    Arrival Time: Jul 20, 2026 16:13:32.336975000 EDT
    UTC Arrival Time: Jul 20, 2026 20:13:32.336975000 UTC
    Epoch Arrival Time: 1784578412.336975000
    [Time shift for this packet: 0.000000000 seconds]
    [Time since reference or first frame: 0.000000000 seconds]
    Frame Number: 1
    Frame Length: 98 bytes (784 bits)
    Capture Length: 98 bytes (784 bits)
    [Frame is marked: False]
    [Frame is ignored: False]
    [Protocols in frame: eth:ethertype:ip:icmp:data]
    Character encoding: ASCII (0)
    [Coloring Rule Name: ICMP]
    [Coloring Rule String: icmp || icmpv6]
∨ Ethernet II, Src: be:16:8a:20:2e:42 (be:16:8a:20:2e:42), Dst: ae:cd:e7:12:64:04 (ae:cd:e7:12:64:04)
  > Destination: ae:cd:e7:12:64:04 (ae:cd:e7:12:64:04)
  > Source: be:16:8a:20:2e:42 (be:16:8a:20:2e:42)
    Type: IPv4 (0x0800)
    [Stream index: 0]
∨ Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 84
    Identification: 0x01bc (444)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: ICMP (1)
    Header Checksum: 0x24eb [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.0.0.1
    Destination Address: 10.0.0.2
    [Stream index: 0]
∨ Internet Control Message Protocol
    Type: Echo (ping) request (8)
    Code: 0
    Checksum: 0xedf8 [correct]
    [Checksum Status: Good]
    Identifier (BE): 36 (0x0024)
    Identifier (LE): 9216 (0x2400)
    Sequence Number (BE): 1 (0x0001)
    Sequence Number (LE): 256 (0x0100)
    [Response frame: 2]
  > ICMP Data: 6c815e6a000000007b2305000000000010111213141516171819 1a1b1c1d1e1f202122232425262728292a2b2c2d
```

Mutest Result:

```
INFO: NUMBER     STAT   TARGET   TIME  DESCRIPTION
INFO: ----------------------------------------------------------------------
INFO:
INFO: 1. Objective 3: PREOF over static routes, no FRR.
INFO:
INFO: 1.1. The DNT replication engine is running on both edge nodes
INFO: 1.1.1      PASS  bridgeA   0.01  A dnt process is running with bridgeA.ini
```

```
INFO: 1.1.2      PASS  bridgeB  0.009  A dnt process is running with bridgeB.ini
INFO:
INFO: 1.2. No FRR or any other routing daemon exists anywhere in the lab
INFO: 1.2.1      PASS  bridgeA  0.008  No zebra, ospfd or frr process is running
INFO:
INFO: 1.3. Every path-facing link is at MTU 1600 to fit the tunnel headers
INFO: 1.3.1      PASS  bridgeA  0.005  eth1 is set to MTU 1600
INFO: 1.3.2      PASS  bridgeA  0.005  eth2 is set to MTU 1600
INFO: 1.3.3      PASS bridge1A  0.005  eth0 is set to MTU 1600
INFO: 1.3.4      PASS bridge1A  0.005  eth1 is set to MTU 1600
INFO: 1.3.5      PASS bridge1B  0.004  eth0 is set to MTU 1600
INFO: 1.3.6      PASS bridge1B  0.004  eth1 is set to MTU 1600
INFO: 1.3.7      PASS bridge2A  0.004  eth0 is set to MTU 1600
INFO: 1.3.8      PASS bridge2A  0.004  eth1 is set to MTU 1600
INFO: 1.3.9      PASS bridge2B  0.004  eth0 is set to MTU 1600
INFO: 1.3.10     PASS bridge2B  0.004  eth1 is set to MTU 1600
INFO: 1.3.11     PASS  bridgeB  0.004  eth1 is set to MTU 1600
INFO: 1.3.12     PASS  bridgeB  0.004  eth2 is set to MTU 1600
INFO:
INFO: 1.4. STATIC ROUTES. Both tunnel paths are pinned by hand, not learned
INFO: 1.4.1      PASS  bridgeA  0.004  Tunnel endpoint 10.1.3.2 routes out eth1, over
Path 1
INFO: 1.4.2      PASS  bridgeA  0.005  Tunnel endpoint 10.2.3.2 routes out eth2, over
Path 2
INFO: 1.4.3      PASS  bridgeB  0.004  The return route to 10.1.1.1 leaves bridgeB on
eth1
INFO: 1.4.4      PASS  bridgeB  0.004  The return route to 10.2.1.1 leaves bridgeB on
eth2
INFO:
INFO: 1.5. Both paths are reachable end to end before any tunnelling is involved
INFO: 1.5.1      PASS  bridgeA    1.0  Ping to 10.1.3.2 succeeds across Path 1
INFO: 1.5.2      PASS  bridgeA    1.0  Ping to 10.2.3.2 succeeds across Path 2
INFO: 1.5.3      PASS  bridgeA    1.0  Traceroute to 10.1.3.2 goes through bridge1A,
as expected
INFO: 1.5.4      PASS  bridgeA    1.0  Traceroute to 10.2.3.2 goes through bridge2A,
as expected
INFO:
INFO: 1.6. Host ARP entries are pinned so replicated ARP stays out of the captures
INFO: 1.6.1      PASS       h1    1.1  Warm-up ping from h1 to h2 got 2 of 2 replies
INFO: 1.6.2      PASS       h1    0.0  The warm-up ping produced no duplicate replies
INFO:
INFO: 1.7. Captures are running on both paths and h1 sends 10 pings
INFO: 1.7.1      PASS bridge1A    0.5  tcpdump listening on eth1 for the path1
capture
INFO: 1.7.2      PASS bridge2A    0.5  tcpdump listening on eth1 for the path2
capture
```

```
INFO: 1.7.3      PASS       h2    0.5  tcpdump listening on eth0 for the listener
capture
INFO: 1.7.4      PASS       h1    9.1  h1 received all 10 ICMP replies
INFO: 1.7.5      PASS       h1    0.0  h1 reported 0% packet loss
INFO: 1.7.6      PASS       h1    0.0  h1 saw no duplicates, so elimination worked
INFO: 1.7.7      PASS bridge1A    2.5  The path1 capture flushed to disk and stopped
INFO: 1.7.8      PASS bridge2A    0.5  The path2 capture flushed to disk and stopped
INFO: 1.7.9      PASS       h2    0.5  The listener capture flushed to disk and
stopped
INFO:
INFO: 1.8. REPLICATION. Both paths carry their own copy of every packet
INFO: 1.8.1      PASS bridge1A   0.01  Count Path 1 packets travelling toward bridgeB
INFO: 1.8.2      PASS bridge2A  0.007  Count Path 2 packets travelling toward bridgeB
INFO: 1.8.3      PASS bridge1A    0.0  Path 1 carried 80 packets toward bridgeB
INFO: 1.8.4      PASS bridge2A    0.0  Path 2 carried 80 packets toward bridgeB
INFO: 1.8.5      PASS             0.0  Both paths carried the same number of copies,
80 and 80
INFO:
INFO: 1.9. SEPARATION. Each path carries only its own MPLS label, never the other
INFO: 1.9.1      PASS bridge1A  0.007  Count Path 1 packets carrying MPLS label 100
INFO: 1.9.2      PASS bridge2A  0.008  Count Path 2 packets carrying MPLS label 200
INFO: 1.9.3      PASS bridge1A    0.0  Path 1 carried 160 label 100 packets, counting
both directions
INFO: 1.9.4      PASS bridge2A    0.0  Path 2 carried 160 label 200 packets, counting
both directions
INFO: 1.9.5      PASS bridge1A  0.007  No label 200 packet ever appeared on Path 1
INFO: 1.9.6      PASS bridge2A  0.006  No label 100 packet ever appeared on Path 2
INFO:
INFO: 1.10. ELIMINATION. The listener receives one copy of each packet, not two
INFO: 1.10.1     PASS       h2  0.008  Count ICMP echo requests arriving at the
listener
INFO: 1.10.2     PASS       h2  0.007  Count ICMP echo replies sent back by the
listener
INFO: 1.10.3     PASS       h2 0.0001  h2 received 10 requests, one per ping (10)
INFO: 1.10.4     PASS       h2    0.0  h2 sent 10 replies, one per request (10)
INFO: 1.10.5     PASS       h2    0.0  160 packets crossed the core but only 10
reached h2
INFO:
INFO: 1.11. PATH 1 DOWN. Delivery survives, but the route needs repairing by hand
INFO: 1.11.1     PASS  bridgeA    1.0  Path 1 is genuinely broken, 10.1.3.2 no longer
answers
INFO: 1.11.2     PASS       h1    4.1  0% loss with Path 1 down, the second copy
carries it
INFO: 1.11.3     PASS       h1    0.0  No duplicate replies while Path 1 is down
INFO: 1.11.4     PASS  bridgeA    1.0  Path 1 works again, but only after the route
was re-added by hand
INFO: 1.11.5     PASS       h1    4.0  0% loss with both paths healthy again
```

```
INFO: 1.11.6     PASS       h1    0.0  No duplicate replies with both paths up
INFO:
INFO: run stats: 56 steps, 56 pass, 0 fail, 0 abort, 32.86s elapsed
INFO: ----------------------------------------------------------------------
INFO:  PASS  1:mutest_obj3_preof
INFO: ----------------------------------------------------------------------
INFO: END RUN: 1 test scripts, 1 passed, 0 failed
```

7. OBJECTIVE 4

Add dynamic routing with FRR/OSPF to test the PREOF scenario

Objective 3 proved PREOF working over paths configured. Every router held a static route, a forwarding entry hardcoded. This objective deletes all of them and lets the network compute the paths itself using FRR's OSPFD.
The goal is to prove that OSPF computes the same disjoint paths the static routes specified by hand, with no metric tuning, so the deterministic service is unaffected by the change.

Steps to achieve Objective 4:

Objective 3's static routes replaced by FRR running OSPF, everything else held constant. OSPF reached Full on every link and computed one route to each far edge address

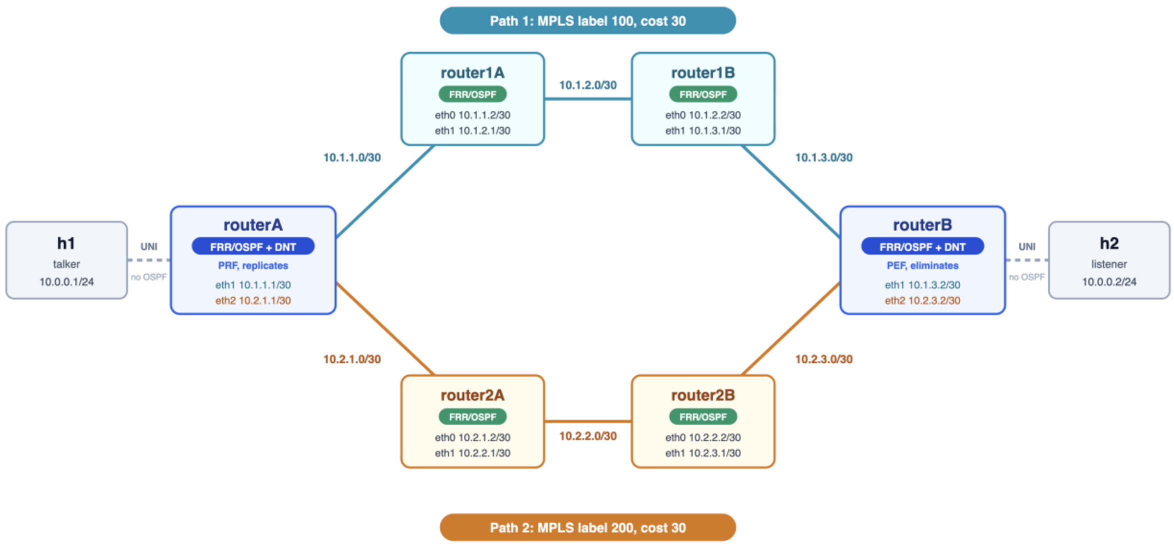
Here is a quick overview of this Objective:

https://docs.google.com/presentation/d/16R1r10A8BYRtZR8U_bsUYTQE4Xk5cZYINzA5Lpb0iiQ/edit?usp=sharing

Step 1. Identify the topology to build:

This objective builds its own topology rather than using a supplied example, so the shape has to be decided before anything is configured. It needs eight nodes: two end hosts, two edge routers running DNT, and four intermediate routers arranged as two separate chains of two. (same as objective 3)

The requirement driving the shape is that the two copies must have two separate paths available to them. Two chains of two intermediate routers give two paths that so a break anywhere on one path leaves the other untouched.



Step 2. Prepare the environment:

Install FRR. FRR provides the two routing daemons this objective needs: zebra, which owns the Linux routing table and installs the actual routes, and OSPFD, which speaks OSPF and tells zebra what it has learned. Build it from source following the FRR developer guide:

```
git clone https://github.com/FRRouting/frr.git
cd frr
./bootstrap.sh
./configure
make
sudo make install
```

The daemons install to /usr/lib/frr, which is not on the shell's PATH, so they are always started by full path. vtysh, the tool used to query them, does land on PATH and is run by name.

Then create a working directory on the VM to hold everything for Objective 4, for example ~/lab/ospf-topo. The paths inside munet.yaml point at this directory, so the whole scenario stays self-contained.

```
mkdir -p ~/lab/ospf-topo
cd ~/lab/ospf-topo
```

Then create ten files inside it:

- munet.yaml: the topology, all eight nodes with their links, addresses, FRR launch, and the DNT launch on each edge.
- munet-phase1.yaml: the same topology with DNT switched off, so the routing can be checked on its own first.
- bridgeA.ini: the DNT PREOF config for Edge Router 1.

- bridgeB.ini: the DNT PREOF config for Edge Router 2.
- bridgeA.conf, bridge1A.conf, bridge1B.conf, bridge2A.conf, bridge2B.conf, bridgeB.conf: one OSPF config per router, six in total.

Populate each file with the contents given in the config listings; they are not repeated here. The directory ends up as:

```
~/lab/ospf-topo/munet.yaml
                munet-phase1.yaml
                routerA.ini
                routerB.ini
                routerA.conf
                router1A.conf
                router1B.conf
                router2A.conf
                router2B.conf
                routerB.conf
```

The whole topology is one munet YAML file. It declares eight point-to-point networks and the eight nodes, and each node's cmd runs at bring-up: the hosts set their address and disable offload, the four intermediates enable IP forwarding, copy their OSPF config into a private FRR pathspace and start zebra then ospfd, and the two edges do the same and then launch DNT. No node adds a static route anywhere.

Configuration:

OSPF:

```
hostname bridge1A
log file /tmp/frr-bridge1A.log informational
!
interface eth0
```

```
 ip ospf network point-to-point
 ip ospf cost 10
!
interface eth1
 ip ospf network point-to-point
 ip ospf cost 10
!
router ospf
 ospf router-id 10.1.1.2
 network 10.1.1.0/30 area 0
 network 10.1.2.0/30 area 0
!
```

Munet:

```
topology:
  networks:
  - name: link-h1
  - name: net1A
  - name: net1B
  - name: net1C
  - name: net2A
  - name: net2B
  - name: net2C
  - name: link-h2

  nodes:
  - name: h1
    cmd: |
      ip addr add 10.0.0.1/24 dev eth0
      ip link set eth0 up
      ethtool -K eth0 rxvlan off txvlan off tx off rx off
      bash
    connections:
    - to: link-h1

  - name: routerA
    cmd: |
      sysctl -w net.ipv4.ip_forward=1
      ip link set eth0 up
      ip link set eth1 mtu 1600 up
      ip link set eth2 mtu 1600 up
      ip addr add 10.1.1.1/30 dev eth1
      ip addr add 10.2.1.1/30 dev eth2
      FRRBIN=/usr/lib/frr; [ -x $FRRBIN/zebra ] || FRRBIN=/usr/local/sbin
      mkdir -p /etc/frr/routerA /var/run/frr/routerA
```

```
    cp /home/francis/lab/ospf-topo/routerA.conf /etc/frr/routerA/frr.conf
    : > /etc/frr/routerA/vtysh.conf
    chown -R frr:frr /etc/frr/routerA /var/run/frr/routerA
    $FRRBIN/zebra -N routerA -d -f /etc/frr/routerA/frr.conf
    sleep 1
    $FRRBIN/ospfd -N routerA -d -f /etc/frr/routerA/frr.conf
    /usr/local/bin/dnt /home/francis/lab/ospf-topo/routerA.ini
  connections:
  - to: link-h1
  - to: net1A
  - to: net2A

- name: router1A
  cmd: |
    sysctl -w net.ipv4.ip_forward=1
    ip link set eth0 mtu 1600 up
    ip link set eth1 mtu 1600 up
    ip addr add 10.1.1.2/30 dev eth0
    ip addr add 10.1.2.1/30 dev eth1
    FRRBIN=/usr/lib/frr; [ -x $FRRBIN/zebra ] || FRRBIN=/usr/local/sbin
    mkdir -p /etc/frr/router1A /var/run/frr/router1A
    cp /home/francis/lab/ospf-topo/router1A.conf /etc/frr/router1A/frr.conf
    : > /etc/frr/router1A/vtysh.conf
    chown -R frr:frr /etc/frr/router1A /var/run/frr/router1A
    $FRRBIN/zebra -N router1A -d -f /etc/frr/router1A/frr.conf
    sleep 1
    $FRRBIN/ospfd -N router1A -d -f /etc/frr/router1A/frr.conf
    bash
  connections:
  - to: net1A
  - to: net1B

- name: router1B
  cmd: |
    sysctl -w net.ipv4.ip_forward=1
    ip link set eth0 mtu 1600 up
    ip link set eth1 mtu 1600 up
    ip addr add 10.1.2.2/30 dev eth0
    ip addr add 10.1.3.1/30 dev eth1
    FRRBIN=/usr/lib/frr; [ -x $FRRBIN/zebra ] || FRRBIN=/usr/local/sbin
    mkdir -p /etc/frr/router1B /var/run/frr/router1B
    cp /home/francis/lab/ospf-topo/router1B.conf /etc/frr/router1B/frr.conf
    : > /etc/frr/router1B/vtysh.conf
    chown -R frr:frr /etc/frr/router1B /var/run/frr/router1B
    $FRRBIN/zebra -N router1B -d -f /etc/frr/router1B/frr.conf
    sleep 1
```

```
    $FRRBIN/ospfd -N router1B -d -f /etc/frr/router1B/frr.conf
    bash
  connections:
  - to: net1B
  - to: net1C

- name: router2A
  cmd: |
    sysctl -w net.ipv4.ip_forward=1
    ip link set eth0 mtu 1600 up
    ip link set eth1 mtu 1600 up
    ip addr add 10.2.1.2/30 dev eth0
    ip addr add 10.2.2.1/30 dev eth1
    FRRBIN=/usr/lib/frr; [ -x $FRRBIN/zebra ] || FRRBIN=/usr/local/sbin
    mkdir -p /etc/frr/router2A /var/run/frr/router2A
    cp /home/francis/lab/ospf-topo/router2A.conf /etc/frr/router2A/frr.conf
    : > /etc/frr/router2A/vtysh.conf
    chown -R frr:frr /etc/frr/router2A /var/run/frr/router2A
    $FRRBIN/zebra -N router2A -d -f /etc/frr/router2A/frr.conf
    sleep 1
    $FRRBIN/ospfd -N router2A -d -f /etc/frr/router2A/frr.conf
    bash
  connections:
  - to: net2A
  - to: net2B

- name: router2B
  cmd: |
    sysctl -w net.ipv4.ip_forward=1
    ip link set eth0 mtu 1600 up
    ip link set eth1 mtu 1600 up
    ip addr add 10.2.2.2/30 dev eth0
    ip addr add 10.2.3.1/30 dev eth1
    FRRBIN=/usr/lib/frr; [ -x $FRRBIN/zebra ] || FRRBIN=/usr/local/sbin
    mkdir -p /etc/frr/router2B /var/run/frr/router2B
    cp /home/francis/lab/ospf-topo/router2B.conf /etc/frr/router2B/frr.conf
    : > /etc/frr/router2B/vtysh.conf
    chown -R frr:frr /etc/frr/router2B /var/run/frr/router2B
    $FRRBIN/zebra -N router2B -d -f /etc/frr/router2B/frr.conf
    sleep 1
    $FRRBIN/ospfd -N router2B -d -f /etc/frr/router2B/frr.conf
    bash
  connections:
  - to: net2B
  - to: net2C
```

```
  - name: routerB
    cmd: |
      sysctl -w net.ipv4.ip_forward=1
      ip link set eth0 up
      ip link set eth1 mtu 1600 up
      ip link set eth2 mtu 1600 up
      ip addr add 10.1.3.2/30 dev eth1
      ip addr add 10.2.3.2/30 dev eth2
      FRRBIN=/usr/lib/frr; [ -x $FRRBIN/zebra ] || FRRBIN=/usr/local/sbin
      mkdir -p /etc/frr/routerB /var/run/frr/routerB
      cp /home/francis/lab/ospf-topo/routerB.conf /etc/frr/routerB/frr.conf
      : > /etc/frr/routerB/vtysh.conf
      chown -R frr:frr /etc/frr/routerB /var/run/frr/routerB
      $FRRBIN/zebra -N routerB -d -f /etc/frr/routerB/frr.conf
      sleep 1
      $FRRBIN/ospfd -N routerB -d -f /etc/frr/routerB/frr.conf
      /usr/local/bin/dnt /home/francis/lab/ospf-topo/routerB.ini
    connections:
    - to: link-h2
    - to: net1C
    - to: net2C

  - name: h2
    cmd: |
      ip addr add 10.0.0.2/24 dev eth0
      ip link set eth0 up
      ethtool -K eth0 rxvlan off txvlan off tx off rx off
      bash
    connections:
    - to: link-h2
```

DNT BridgeA:

```
[interfaces]
uni = eth iface=eth0
uni:streams = prio-compound compound

nni1_in = udp-in iface=eth1 ipv=4
nni1_in:streams = member1 prio-members
nni2_in = udp-in iface=eth2 ipv=4
nni2_in:streams = member2 prio-members

nni1_out = udp-out iface=eth1 dstip=10.1.3.2
nni2_out = udp-out iface=eth2 dstip=10.2.3.2

[objects]
```

```
prf = Replicate
gen = SeqGen InitSeqStart=0
pef = SeqRcvy

genprio = SeqGen InitSeqStart=0
prfprio = Replicate
pefprio = SeqRcvy

[streams]
compound:packet = eth, cvlan
compound:match = cvlan vid=0
compound:actions = gen, edit cvlan.vid=99, before eth add mpls, after mpls add dcw,
writeseq dcw, prf prf-member1 prf-member2

prf-member1 = edit mpls.label=100 mpls.bos=1, send nni1_out
prf-member2 = edit mpls.label=200 mpls.bos=1, send nni2_out

member1:packet = mpls, dcw, eth, cvlan
member1:match = mpls label=100
member1:actions = readseq dcw, pef pef-compound

member2:packet = mpls, dcw, eth, cvlan
member2:match = mpls label=200
member2:actions = readseq dcw, pef pef-compound

pef-compound = del dcw, del mpls, del cvlan, send uni

prio-compound:packet = eth, cvlan
prio-compound:match = cvlan vid=0 pcp=6
prio-compound:actions = genprio, edit cvlan.vid=200, before eth add mpls, after mpls
add dcw, writeseq dcw, prfprio prfprio-member1 prfprio-member2

prfprio-member1 = edit mpls.label=500 mpls.bos=1, send nni1_out
prfprio-member2 = edit mpls.label=500 mpls.bos=1, send nni2_out

prio-members:packet = mpls, dcw, eth, cvlan
prio-members:match = mpls label=500
prio-members:actions = readseq dcw, pefprio pefprio-compound

pefprio-compound = del dcw, del mpls, edit cvlan.vid=0, send uni
```

DNT BridgeB:

```
[interfaces]
uni = eth iface=eth0
uni:streams = prio-compound compound
```

```
nni1_in = udp-in iface=eth1 ipv=4
nni1_in:streams = member1 prio-members
nni2_in = udp-in iface=eth2 ipv=4
nni2_in:streams = member2 prio-members

nni1_out = udp-out iface=eth1 dstip=10.1.1.1
nni2_out = udp-out iface=eth2 dstip=10.2.1.1

[objects]
prf = Replicate
gen = SeqGen InitSeqStart=0
pef = SeqRcvy

genprio = SeqGen InitSeqStart=0
prfprio = Replicate
pefprio = SeqRcvy

[streams]
compound:packet = eth, cvlan
compound:match = cvlan vid=0
compound:actions = gen, edit cvlan.vid=99, before eth add mpls, after mpls add dcw,
writeseq dcw, prf prf-member1 prf-member2

prf-member1 = edit mpls.label=100 mpls.bos=1, send nni1_out
prf-member2 = edit mpls.label=200 mpls.bos=1, send nni2_out

member1:packet = mpls, dcw, eth, cvlan
member1:match = mpls label=100
member1:actions = readseq dcw, pef pef-compound

member2:packet = mpls, dcw, eth, cvlan
member2:match = mpls label=200
member2:actions = readseq dcw, pef pef-compound

pef-compound = del dcw, del mpls, del cvlan, send uni

prio-compound:packet = eth, cvlan
prio-compound:match = cvlan vid=0 pcp=6
prio-compound:actions = genprio, edit cvlan.vid=200, before eth add mpls, after mpls
add dcw, writeseq dcw, prfprio prfprio-member1 prfprio-member2

prfprio-member1 = edit mpls.label=500 mpls.bos=1, send nni1_out
prfprio-member2 = edit mpls.label=500 mpls.bos=1, send nni2_out

prio-members:packet = mpls, dcw, eth, cvlan
prio-members:match = mpls label=500
prio-members:actions = readseq dcw, pefprio pefprio-compound
```

```
pefprio-compound = del dcw, del mpls, edit cvlan.vid=0, send uni
```

Step 5. Bring up the topology:

- Become root first, then start the terminal session manager, in that order. sudo does not pass through the variable that tells munet a session manager is available, and without it munet cannot open its per-node windows:

  - `sudo -i`

- Start the topology from the directory, opening one window per router. Commands are typed into each router's own window because munet's built-in prompt does not work on this machine's version of Python:

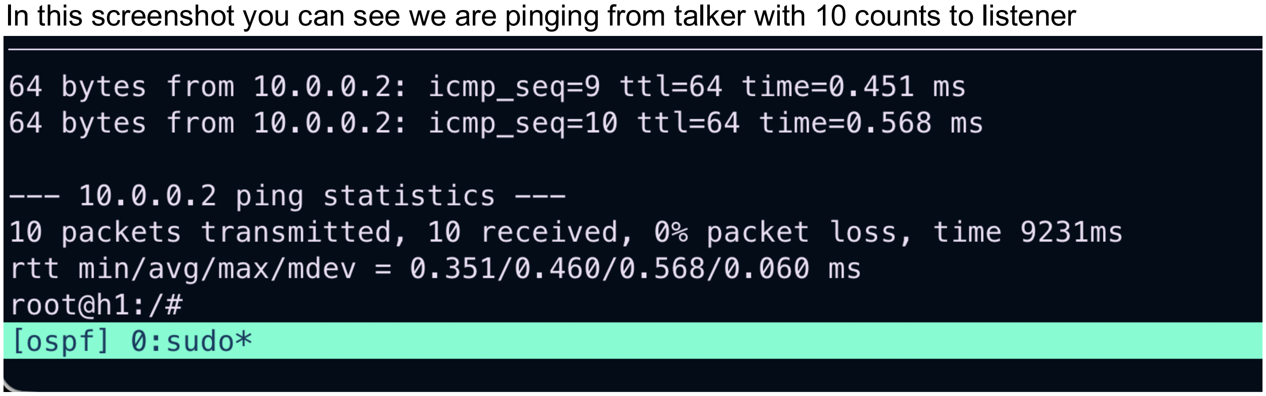
  - `munet -c munet-phase1.yaml --shell routerA,router1A,router1B,router2A,router2B,routerB`

OSPF greets its neighbours every 10 seconds and waits 40 before giving up on one, so from a cold start allow a full 40 seconds before concluding anything is wrong.

Steps to validate this objective from the shell:

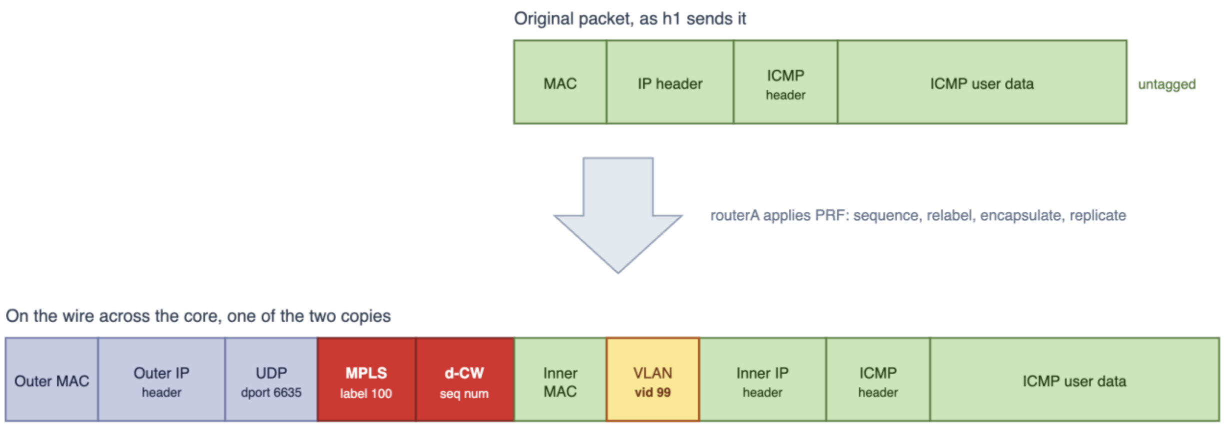
Validating the Application Connectivity:

- In this screenshot you can see we are pinging from talker with 10 counts to listener



Expected Encapsulation of the packet:

This packet is a logical representation of the original ICMP packet initiated by the talker (h1) and the expected encapsulation on the interface Router1A eth1.



Life of a Packet:

Packet capture on talker eth0:

- Here is the packet before DNT touches it. Plain Ethernet, IPv4, ICMP, untagged, nothing in between. 10.0.0.1 to 10.0.0.2, TTL 64, 98 bytes, sequences 1 to 10.

```
No.        Time          Source              Destination          Protocol  Length  Info
> Frame 1: Packet, 98 bytes on wire (784 bits), 98 bytes captured (784 bits)
v Ethernet II, Src: 32:a7:06:68:c1:29 (32:a7:06:68:c1:29), Dst: 6e:b7:49:0e:39:cf (6e:b7:49:0e:39:cf)
  > Destination: 6e:b7:49:0e:39:cf (6e:b7:49:0e:39:cf)
  > Source: 32:a7:06:68:c1:29 (32:a7:06:68:c1:29)
    Type: IPv4 (0x0800)
    [Stream index: 0]
v Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 84
    Identification: 0x62d2 (25298)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: ICMP (1)
    Header Checksum: 0xc3d4 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.0.0.1
    Destination Address: 10.0.0.2
    [Stream index: 0]
v Internet Control Message Protocol
    Type: Echo (ping) request (8)
    Code: 0
    Checksum: 0x08b3 [correct]
    [Checksum Status: Good]
    Identifier (BE): 629 (0x0275)
    Identifier (LE): 29954 (0x7502)
    Sequence Number (BE): 1 (0x0001)
    Sequence Number (LE): 256 (0x0100)
    [Response frame: 2]
  > ICMP Data: a7356a6a000000001a6402000000000010111213141516171819 1a1b1c1d1e1f202122232425262728292a2b2c2d2
```

Packet capture on router1A eth1:

- Here is a packet captured on Path 1. As you can see it is the same talker to listener ICMP, now encapsulated with MPLS label = 100 and control word sequence = 14, inside a UDP port 6635 tunnel. The label matches prf-member1 in the interface config. 20 packets, 152 bytes each, every one label 100.

```
No.    Time    Source    Destination    Protocol  Lengtr  Info
∨ Ethernet II, Src: 6a:51:4f:6d:9e:06 (6a:51:4f:6d:9e:06), Dst: 8a:95:5d:ac:7b:cd (8a:95:5d:ac:7b:cd)
  > Destination: 8a:95:5d:ac:7b:cd (8a:95:5d:ac:7b:cd)
  > Source: 6a:51:4f:6d:9e:06 (6a:51:4f:6d:9e:06)
    Type: IPv4 (0x0800)
    [Stream index: 0]
∨ Internet Protocol Version 4, Src: 10.1.1.1, Dst: 10.1.3.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 138
    Identification: 0x0000 (0)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 63
    Protocol: UDP (17)
    Header Checksum: 0x235f [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.1.1.1
    Destination Address: 10.1.3.2
    [Stream index: 0]
∨ User Datagram Protocol, Src Port: 52875, Dst Port: 6635
    Source Port: 52875
    Destination Port: 6635
    Length: 118
    Checksum: 0x188c [unverified]
    [Checksum Status: Unverified]
    [Stream index: 0]
    [Stream Packet Number: 1]
  > [Timestamps]
    UDP payload (110 bytes)
∨ MultiProtocol Label Switching Header, Label: 100, Exp: 0, S: 1, TTL: 0
    0000 0000 0000 0110 0100 .... .... .... = MPLS Label: 100 (0x00064)
    .... .... .... .... .... 000. .... .... = MPLS Experimental Bits: 0
    .... .... .... .... .... ...1 .... .... = MPLS Bottom Of Label Stack: 1
    .... .... .... .... .... .... 0000 0000 = MPLS TTL: 0
∨ PW Ethernet Control Word
    Sequence Number: 14
∨ Ethernet II, Src: 32:a7:06:68:c1:29 (32:a7:06:68:c1:29), Dst: 6e:b7:49:0e:39:cf (6e:b7:49:0e:39:cf)
  > Destination: 6e:b7:49:0e:39:cf (6e:b7:49:0e:39:cf)
  > Source: 32:a7:06:68:c1:29 (32:a7:06:68:c1:29)
    Type: 802.1Q Virtual LAN (0x8100)
    [Stream index: 1]
∨ 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 99
    000. .... .... .... = Priority: Best Effort (default) (0)
    ...0 .... .... .... = DEI: Ineligible
    .... 0000 0110 0011 = ID: 99
    Type: IPv4 (0x0800)
∨ Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 84
```

Packet capture on router2A eth1:

- Same thing here. Same stack. Three fields differ: label 200, outer addresses 10.2.1.1 to 10.2.3.2, source port 46245. Those match prf-member2 and nni2_out in the config.

```
No.    Time    Source    Destination    Protocol    Length    Info
v Ethernet II, Src: 16:1b:10:c9:8b:2c (16:1b:10:c9:8b:2c), Dst: 3e:2c:09:18:98:36 (3e:2c:09:18:98:36)
  > Destination: 3e:2c:09:18:98:36 (3e:2c:09:18:98:36)
  > Source: 16:1b:10:c9:8b:2c (16:1b:10:c9:8b:2c)
    Type: IPv4 (0x0800)
    [Stream index: 0]
v Internet Protocol Version 4, Src: 10.2.1.1, Dst: 10.2.3.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 138
    Identification: 0x0000 (0)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 63
    Protocol: UDP (17)
    Header Checksum: 0x235d [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.2.1.1
    Destination Address: 10.2.3.2
    [Stream index: 0]
v User Datagram Protocol, Src Port: 46245, Dst Port: 6635
    Source Port: 46245
    Destination Port: 6635
    Length: 118
    Checksum: 0x188e [unverified]
    [Checksum Status: Unverified]
    [Stream index: 0]
    [Stream Packet Number: 1]
  > [Timestamps]
    UDP payload (110 bytes)
v MultiProtocol Label Switching Header, Label: 200, Exp: 0, S: 1, TTL: 0
    0000 0000 0000 1100 1000 .... .... .... = MPLS Label: 200 (0x000c8)
    .... .... .... .... .... 000. .... .... = MPLS Experimental Bits: 0
    .... .... .... .... .... ...1 .... .... = MPLS Bottom Of Label Stack: 1
    .... .... .... .... .... .... 0000 0000 = MPLS TTL: 0
v PW Ethernet Control Word
    Sequence Number: 14
v Ethernet II, Src: 32:a7:06:68:c1:29 (32:a7:06:68:c1:29), Dst: 6e:b7:49:0e:39:cf (6e:b7:49:0e:39:cf)
  > Destination: 6e:b7:49:0e:39:cf (6e:b7:49:0e:39:cf)
  > Source: 32:a7:06:68:c1:29 (32:a7:06:68:c1:29)
    Type: 802.1Q Virtual LAN (0x8100)
    [Stream index: 1]
v 802.1Q Virtual LAN, PRI: 0, DEI: 0, ID: 99
    000. .... .... .... = Priority: Best Effort (default) (0)
    ...0 .... .... .... = DEI: Ineligible
    .... 0000 0110 0011 = ID: 99
    Type: IPv4 (0x0800)
v Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 84
```

Packet capture on listener eth0:

- Here is the packet after elimination. Back to plain Ethernet, IPv4, ICMP, untagged, TTL 64, 98 bytes, sequences 1 to 10 once each. Byte identical to what the talker sent. Everything the near edge added is gone: outer headers, MPLS label, control word, VLAN 99.

```
> Frame 1: Packet, 98 bytes on wire (784 bits), 98 bytes captured (784 bits)
v Ethernet II, Src: 32:a7:06:68:c1:29 (32:a7:06:68:c1:29), Dst: 6e:b7:49:0e:39:cf (6e:b7:49:0e:39:cf)
  > Destination: 6e:b7:49:0e:39:cf (6e:b7:49:0e:39:cf)
  > Source: 32:a7:06:68:c1:29 (32:a7:06:68:c1:29)
    Type: IPv4 (0x0800)
    [Stream index: 0]
v Internet Protocol Version 4, Src: 10.0.0.1, Dst: 10.0.0.2
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  > Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 84
    Identification: 0x62d2 (25298)
  > 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 64
    Protocol: ICMP (1)
    Header Checksum: 0xc3d4 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 10.0.0.1
    Destination Address: 10.0.0.2
    [Stream index: 0]
v Internet Control Message Protocol
    Type: Echo (ping) request (8)
    Code: 0
    Checksum: 0x08b3 [correct]
    [Checksum Status: Good]
    Identifier (BE): 629 (0x0275)
    Identifier (LE): 29954 (0x7502)
    Sequence Number (BE): 1 (0x0001)
    Sequence Number (LE): 256 (0x0100)
    [Response frame: 2]
  > ICMP Data: a7356a6a000000001a640200000000000101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d2
```

Mutest:

```
INFO: NUMBER     STAT   TARGET   TIME  DESCRIPTION
INFO: ---------------------------------------------------------------------
INFO:
INFO: 1. Objective 4: PREOF over OSPF, equal cost then Path 2 re-costed.
INFO:
INFO: 1.1. The DNT replication engine is running on both edge nodes
INFO: 1.1.1      PASS  bridgeA    1.0  A dnt process is running with bridgeA.ini
```

```
INFO: 1.1.2      PASS  bridgeB  0.007  A dnt process is running with bridgeB.ini
INFO:
INFO: 1.2. OSPF is running on all six routers and every adjacency is Full
INFO: 1.2.1      PASS  bridgeA  0.007  The zebra routing daemon is running
INFO: 1.2.2      PASS  bridgeA  0.006  The ospfd OSPF daemon is running
INFO: 1.2.3      PASS bridge1A  0.007  The zebra routing daemon is running
INFO: 1.2.4      PASS bridge1A  0.007  The ospfd OSPF daemon is running
INFO: 1.2.5      PASS bridge1B  0.007  The zebra routing daemon is running
INFO: 1.2.6      PASS bridge1B  0.007  The ospfd OSPF daemon is running
INFO: 1.2.7      PASS bridge2A  0.006  The zebra routing daemon is running
INFO: 1.2.8      PASS bridge2A  0.006  The ospfd OSPF daemon is running
INFO: 1.2.9      PASS bridge2B  0.006  The zebra routing daemon is running
INFO: 1.2.10     PASS bridge2B  0.006  The ospfd OSPF daemon is running
INFO: 1.2.11     PASS  bridgeB  0.006  The zebra routing daemon is running
INFO: 1.2.12     PASS  bridgeB  0.006  The ospfd OSPF daemon is running
INFO: 1.2.13     PASS  bridgeA   10.4  Both OSPF neighbours have reached Full state
INFO: 1.2.14     PASS bridge1A   0.02  Both OSPF neighbours have reached Full state
INFO: 1.2.15     PASS bridge1B   0.02  Both OSPF neighbours have reached Full state
INFO: 1.2.16     PASS bridge2A    4.7  Both OSPF neighbours have reached Full state
INFO: 1.2.17     PASS bridge2B   0.02  Both OSPF neighbours have reached Full state
INFO: 1.2.18     PASS  bridgeB   0.02  Both OSPF neighbours have reached Full state
INFO:
INFO: 1.3. Every path-facing link is at MTU 1600 to fit the tunnel headers
INFO: 1.3.1      PASS  bridgeA  0.005  eth1 is set to MTU 1600
INFO: 1.3.2      PASS  bridgeA  0.005  eth2 is set to MTU 1600
INFO: 1.3.3      PASS bridge1A  0.005  eth0 is set to MTU 1600
INFO: 1.3.4      PASS bridge1A  0.005  eth1 is set to MTU 1600
INFO: 1.3.5      PASS bridge1B  0.004  eth0 is set to MTU 1600
INFO: 1.3.6      PASS bridge1B  0.004  eth1 is set to MTU 1600
INFO: 1.3.7      PASS bridge2A  0.005  eth0 is set to MTU 1600
INFO: 1.3.8      PASS bridge2A  0.004  eth1 is set to MTU 1600
INFO: 1.3.9      PASS bridge2B  0.004  eth0 is set to MTU 1600
INFO: 1.3.10     PASS bridge2B  0.004  eth1 is set to MTU 1600
INFO: 1.3.11     PASS  bridgeB  0.004  eth1 is set to MTU 1600
INFO: 1.3.12     PASS  bridgeB  0.004  eth2 is set to MTU 1600
INFO:
INFO: 1.4. Both tunnel paths are learned from OSPF, with nothing configured by hand
INFO: 1.4.1      PASS  bridgeA  0.004  No static routes exist, all forwarding comes
from OSPF
INFO: 1.4.2      PASS  bridgeB  0.004  No static routes exist, all forwarding comes
from OSPF
INFO: 1.4.3      PASS  bridgeA  0.004  Path 1 far-side subnet 10.1.3.0/30 was learned
from OSPF
INFO: 1.4.4      PASS  bridgeA  0.004  Path 2 far-side subnet 10.2.3.0/30 was learned
from OSPF
```

```
INFO: 1.4.5      PASS  bridgeA  0.004  Tunnel endpoint 10.1.3.2 routes out eth1, over
Path 1
INFO: 1.4.6      PASS  bridgeA    5.1  Tunnel endpoint 10.2.3.2 routes out eth2, over
Path 2
INFO: 1.4.7      PASS  bridgeB  0.004  The return route to 10.1.1.1 leaves bridgeB on
eth1
INFO: 1.4.8      PASS  bridgeB  0.004  The return route to 10.2.1.1 leaves bridgeB on
eth2
INFO:
INFO: 1.5. Both paths are reachable end to end before any tunnelling is involved
INFO: 1.5.1      PASS  bridgeA    1.0  Ping to 10.1.3.2 succeeds across Path 1
INFO: 1.5.2      PASS  bridgeA    1.0  Ping to 10.2.3.2 succeeds across Path 2
INFO: 1.5.3      PASS  bridgeA  0.006  Traceroute to 10.1.3.2 goes through bridge1A,
as expected
INFO: 1.5.4      PASS  bridgeA  0.005  Traceroute to 10.2.3.2 goes through bridge2A,
as expected
INFO:
INFO: 1.6. Host ARP entries are pinned so replicated ARP stays out of the captures
INFO: 1.6.1      PASS       h1    1.1  Warm-up ping from h1 to h2 got 2 of 2 replies
INFO: 1.6.2      PASS       h1    0.0  The warm-up ping produced no duplicate replies
INFO:
INFO: 1.7. EQUAL COST. Captures are running on both paths and h1 sends 10 pings
INFO: 1.7.1      PASS bridge1A    0.5  tcpdump listening on eth1 for the path1
capture
INFO: 1.7.2      PASS bridge2A    0.5  tcpdump listening on eth1 for the path2
capture
INFO: 1.7.3      PASS       h2    0.5  tcpdump listening on eth0 for the listener
capture
INFO: 1.7.4      PASS       h1    9.2  h1 received all 10 ICMP replies
INFO: 1.7.5      PASS       h1    0.0  h1 reported 0% packet loss
INFO: 1.7.6      PASS       h1    0.0  h1 saw no duplicates, so elimination worked
INFO: 1.7.7      PASS bridge1A    2.5  The path1 capture flushed to disk and stopped
INFO: 1.7.8      PASS bridge2A    0.5  The path2 capture flushed to disk and stopped
INFO: 1.7.9      PASS       h2    0.5  The listener capture flushed to disk and
stopped
INFO:
INFO: 1.8. REPLICATION. Both paths carry their own copy of every packet
INFO: 1.8.1      PASS bridge1A   0.01  Count Path 1 packets travelling toward bridgeB
INFO: 1.8.2      PASS bridge2A  0.008  Count Path 2 packets travelling toward bridgeB
INFO: 1.8.3      PASS bridge1A    0.0  Path 1 carried 80 packets toward bridgeB
INFO: 1.8.4      PASS bridge2A    0.0  Path 2 carried 80 packets toward bridgeB
INFO: 1.8.5      PASS             0.0  Both paths carried the same number of copies,
80 and 80
INFO:
INFO: 1.9. SEPARATION. Each path carries only its own MPLS label, never the other
INFO: 1.9.1      PASS bridge1A  0.010  Count Path 1 packets carrying MPLS label 100
INFO: 1.9.2      PASS bridge2A  0.008  Count Path 2 packets carrying MPLS label 200
```

```
INFO: 1.9.3      PASS bridge1A    0.0  Path 1 carried 160 label 100 packets, counting
both directions
INFO: 1.9.4      PASS bridge2A    0.0  Path 2 carried 160 label 200 packets, counting
both directions
INFO: 1.9.5      PASS bridge1A  0.007  No label 200 packet ever appeared on Path 1
INFO: 1.9.6      PASS bridge2A  0.006  No label 100 packet ever appeared on Path 2
INFO:
INFO: 1.10. ELIMINATION. The listener receives one copy of each packet, not two
INFO: 1.10.1     PASS       h2  0.007  Count ICMP echo requests arriving at the
listener
INFO: 1.10.2     PASS       h2  0.007  Count ICMP echo replies sent back by the
listener
INFO: 1.10.3     PASS       h2    0.0  h2 received 10 requests, one per ping (10)
INFO: 1.10.4     PASS       h2    0.0  h2 sent 10 replies, one per request (10)
INFO: 1.10.5     PASS       h2    0.0  160 packets crossed the core but only 10
reached h2
INFO:
INFO: 1.11. PATH 1 FAILS. Delivery survives on member 2, not on the OSPF reroute
INFO: 1.11.1     PASS  bridgeA    0.5  OSPF moved the route for 10.1.3.2 onto eth2 by
itself
INFO: 1.11.2     PASS bridge1A    0.5  tcpdump listening on eth0 for the fail-path1
capture
INFO: 1.11.3     PASS bridge2A    0.5  tcpdump listening on eth1 for the fail-path2
capture
INFO: 1.11.4     PASS bridge1A    6.6  The fail-path1 capture flushed to disk and
stopped
INFO: 1.11.5     PASS bridge2A    0.5  The fail-path2 capture flushed to disk and
stopped
INFO: 1.11.6     PASS bridge1A  0.008  Count packets still arriving at bridge1A from
bridgeA
INFO: 1.11.7     PASS bridge2A  0.008  Count label 200 packets still crossing Path 2
INFO: 1.11.8     PASS       h1    0.0  0% loss with Path 1 down
INFO: 1.11.9     PASS       h1    0.0  No duplicate replies while Path 1 is down
INFO: 1.11.10    PASS bridge2A    0.0  Path 2 still carried 80 label 200 packets
INFO: 1.11.11    PASS  bridgeA    0.0  Member 1 never left bridgeA, 0 packets reached
bridge1A
INFO: 1.11.12    PASS  bridgeA  0.006  Both routes now point out eth2, but member 1
stays pinned to eth1
INFO: 1.11.13    PASS  bridgeA    3.6  OSPF restored the route for 10.1.3.2 with no
manual repair
INFO: 1.11.14    PASS       h1    4.1  0% loss with both paths healthy again
INFO: 1.11.15    PASS       h1    0.0  No duplicate replies with both paths up
INFO:
INFO: 1.12. UNEQUAL COST. Path 2 is re-costed from 10 to 50 while the lab runs
INFO: 1.12.1     PASS  bridgeA    0.1  eth2 now advertises OSPF cost 50
INFO: 1.12.2     PASS bridge2A   0.01  eth0 now advertises OSPF cost 50
INFO: 1.12.3     PASS bridge2A   0.01  eth1 now advertises OSPF cost 50
```

```
INFO: 1.12.4     PASS bridge2B   0.01  eth0 now advertises OSPF cost 50
INFO: 1.12.5     PASS bridge2B   0.01  eth1 now advertises OSPF cost 50
INFO: 1.12.6     PASS  bridgeB   0.01  eth2 now advertises OSPF cost 50
INFO: 1.12.7     PASS  bridgeA    5.1  Tunnel endpoint 10.2.3.2 has moved off eth2
and onto eth1
INFO: 1.12.8     PASS  bridgeB  0.005  The return route to 10.2.1.1 has also moved
onto eth1
INFO: 1.12.9     PASS  bridgeA  0.005  Tunnel endpoint 10.1.3.2 is unchanged, still
out eth1
INFO:
INFO: 1.13. Both endpoints now point down Path 1 and h1 sends 10 more pings
INFO: 1.13.1     PASS bridge1A    0.5  tcpdump listening on eth1 for the cost-path1
capture
INFO: 1.13.2     PASS bridge2A    0.5  tcpdump listening on eth1 for the cost-path2
capture
INFO: 1.13.3     PASS       h2    0.5  tcpdump listening on eth0 for the cost-
listener capture
INFO: 1.13.4     PASS       h1    9.1  h1 still received all 10 ICMP replies
INFO: 1.13.5     PASS       h1    0.0  h1 still reported 0% packet loss
INFO: 1.13.6     PASS bridge1A    2.5  The cost-path1 capture flushed to disk and
stopped
INFO: 1.13.7     PASS bridge2A    0.5  The cost-path2 capture flushed to disk and
stopped
INFO: 1.13.8     PASS       h2    0.5  The cost-listener capture flushed to disk and
stopped
INFO:
INFO: 1.14. REDUNDANCY IS GONE. Only one copy now reaches the wire at all
INFO: 1.14.1     PASS bridge1A  0.007  Count Path 1 packets after the cost change
INFO: 1.14.2     PASS bridge2A  0.007  Count Path 2 packets after the cost change
INFO: 1.14.3     PASS bridge1A  0.007  Count label 100 packets on Path 1 after the
cost change
INFO: 1.14.4     PASS bridge1A  0.007  Count label 200 packets on Path 1 after the
cost change
INFO: 1.14.5     PASS bridge1A    0.0  Path 1 still carried 160 packets, counting
both directions
INFO: 1.14.6     PASS bridge1A    0.0  Path 1 still carried 160 label 100 packets,
both directions
INFO: 1.14.7     PASS bridge2A    0.0  Path 2 carried nothing at all, 0 packets, the
second copy is gone
INFO: 1.14.8     PASS bridge1A    0.0  Label 200 appears nowhere, 0 of them even on
Path 1
INFO:
INFO: 1.15. Delivery still succeeds, on the single surviving copy
INFO: 1.15.1     PASS       h2  0.008  Count ICMP echo requests still arriving at the
listener
INFO: 1.15.2     PASS       h2  0.008  Count ICMP echo replies still sent by the
listener
```

```
INFO: 1.15.3     PASS       h2    0.0  h2 still received all 10 requests (10)
INFO: 1.15.4     PASS       h2    0.0  h2 still sent all 10 replies (10)
INFO:
INFO: 1.16. THE COST OF THAT. The same link failure now loses traffic
INFO: 1.16.1     PASS       h1    5.1  Traffic was lost while OSPF reconverged, 20%
of it
INFO: 1.16.2     PASS  bridgeA  0.003  OSPF pushed 10.2.3.2 back onto eth2 after the
failure
INFO: 1.16.3     PASS       h1    4.1  0% loss once OSPF had finished reconverging
INFO:
INFO: 1.17. CONTROL. Costs go back to 10 and the redundancy returns
INFO: 1.17.1     PASS  bridgeA   0.07  eth2 now advertises OSPF cost 10
INFO: 1.17.2     PASS bridge2A  0.008  eth0 now advertises OSPF cost 10
INFO: 1.17.3     PASS bridge2A  0.008  eth1 now advertises OSPF cost 10
INFO: 1.17.4     PASS bridge2B  0.008  eth0 now advertises OSPF cost 10
INFO: 1.17.5     PASS bridge2B  0.008  eth1 now advertises OSPF cost 10
INFO: 1.17.6     PASS  bridgeB  0.008  eth2 now advertises OSPF cost 10
INFO: 1.17.7     PASS  bridgeA    7.1  Tunnel endpoint 10.1.3.2 is back out eth1
INFO: 1.17.8     PASS  bridgeA  0.004  Tunnel endpoint 10.2.3.2 is back out eth2,
where it started
INFO: 1.17.9     PASS bridge1A    0.5  tcpdump listening on eth1 for the back-path1
capture
INFO: 1.17.10    PASS bridge2A    0.5  tcpdump listening on eth1 for the back-path2
capture
INFO: 1.17.11    PASS bridge1A   11.8  The back-path1 capture flushed to disk and
stopped
INFO: 1.17.12    PASS bridge2A    0.5  The back-path2 capture flushed to disk and
stopped
INFO: 1.17.13    PASS bridge1A  0.009  Count Path 1 packets once the costs were
restored
INFO: 1.17.14    PASS bridge2A  0.008  Count Path 2 packets once the costs were
restored
INFO: 1.17.15    PASS       h1    0.0  h1 reported 0% packet loss with costs back at
10
INFO: 1.17.16    PASS bridge1A    0.0  Path 1 is carrying 160 packets again, both
directions
INFO: 1.17.17    PASS bridge2A    0.0  Path 2 is carrying 160 packets again, both
directions
INFO: 1.17.18    PASS             0.0  Both paths are carrying equally again, 160 and
160
INFO:
INFO: run stats: 136 steps, 136 pass, 0 fail, 0 abort, 107.94s elapsed
INFO: ----------------------------------------------------------------------
INFO:  PASS  1:mutest_obj4_ospf
INFO: ----------------------------------------------------------------------
INFO: END RUN: 1 test scripts, 1 passed, 0 failed
```

8. REFERENCES

12.1 IETF RFCs

RFC 2328: Moy, J., "OSPF Version 2", RFC 2328, DOI 10.17487/RFC2328, April 1998. https://www.rfc-editor.org/rfc/rfc2328.html

RFC 7510: Xu, X., Sheth, N., Yong, L., Callon, R., and D. Black, "Encapsulating MPLS in UDP", RFC 7510, DOI 10.17487/RFC7510, April 2015. https://www.rfc-editor.org/rfc/rfc7510.html

RFC 8655: Finn, N., Thubert, P., Varga, B., and J. Farkas, "Deterministic Networking Architecture", RFC 8655, DOI 10.17487/RFC8655, October 2019. https://www.rfc-editor.org/rfc/rfc8655.html

RFC 8938: Varga, B., Ed., Farkas, J., Berger, L., Malis, A., and S. Bryant, "Deterministic Networking (DetNet) Data Plane Framework", RFC 8938, DOI 10.17487/RFC8938, November 2020. https://www.rfc-editor.org/rfc/rfc8938.html

RFC 8939: Varga, B., Ed., Farkas, J., Berger, L., Fedyk, D., and S. Bryant, "Deterministic Networking (DetNet) Data Plane: IP", RFC 8939, DOI 10.17487/RFC8939, November 2020. https://www.rfc-editor.org/rfc/rfc8939.html

RFC 8964: Varga, B., Ed., Farkas, J., Berger, L., Malis, A., Bryant, S., and J. Korhonen, "Deterministic Networking (DetNet) Data Plane: MPLS", RFC 8964, DOI 10.17487/RFC8964, January 2021. https://www.rfc-editor.org/rfc/rfc8964.html

RFC 9016: Varga, B., Farkas, J., Cummings, R., Jiang, Y., and D. Fedyk, "Flow and Service Information Model for Deterministic Networking (DetNet)", RFC 9016, DOI 10.17487/RFC9016, March 2021. https://www.rfc-editor.org/rfc/rfc9016.html

RFC 9023: Varga, B., Ed., Farkas, J., Malis, A., and S. Bryant, "Deterministic Networking (DetNet) Data Plane: IP over IEEE 802.1 Time-Sensitive Networking (TSN)", RFC 9023, DOI 10.17487/RFC9023, June 2021. https://www.rfc-editor.org/rfc/rfc9023.html

RFC 9024: Varga, B., Ed., Farkas, J., Malis, A., Bryant, S., and D. Fedyk, "Deterministic Networking (DetNet) Data Plane: IEEE 802.1 Time-Sensitive Networking over MPLS", RFC 9024, DOI 10.17487/RFC9024, June 2021. https://www.rfc-editor.org/rfc/rfc9024.html

RFC 9025: Varga, B., Ed., Farkas, J., Berger, L., Malis, A., and S. Bryant, "Deterministic Networking (DetNet) Data Plane: MPLS over UDP/IP", RFC 9025, DOI 10.17487/RFC9025, April 2021. https://www.rfc-editor.org/rfc/rfc9025.html

RFC 9055: Grossman, E., Ed., Mizrahi, T., and A. Hacker, "Deterministic Networking (DetNet) Security Considerations", RFC 9055, DOI 10.17487/RFC9055, June 2021. https://www.rfc-editor.org/rfc/rfc9055.html

RFC 9320: Finn, N., Le Boudec, J., Mohammadpour, E., Zhang, J., and B. Varga, "Deterministic Networking (DetNet) Bounded Latency", RFC 9320, DOI 10.17487/RFC9320, November 2022. https://www.rfc-editor.org/rfc/rfc9320.html

RFC 9546: Mirsky, G., Chen, M., and B. Varga, "Operations, Administration, and Maintenance (OAM) for Deterministic Networking (DetNet) with the MPLS Data Plane", RFC 9546, DOI 10.17487/RFC9546, February 2024. https://www.rfc-editor.org/rfc/rfc9546.html

RFC 9550: Varga, B., Ed., Farkas, J., Kehrer, S., and T. Heer, "Deterministic Networking (DetNet): Packet Ordering Function", RFC 9550, DOI 10.17487/RFC9550, March 2024. https://www.rfc-editor.org/rfc/rfc9550.html

RFC 9566: Varga, B., Farkas, J., and A. Malis, "Deterministic Networking (DetNet) Packet Replication, Elimination, and Ordering Functions (PREOF) via MPLS over UDP/IP", RFC 9566, DOI 10.17487/RFC9566, April 2024. https://www.rfc-editor.org/rfc/rfc9566.html

RFC 9633: Geng, X., Ryoo, Y., Fedyk, D., Rahman, R., and Z. Li, "Deterministic Networking (DetNet) YANG Data Model", RFC 9633, DOI 10.17487/RFC9633, October 2024. https://www.rfc-editor.org/rfc/rfc9633.html

12.2 IEEE standards

IEEE 802.1CB-2017: "IEEE Standard for Local and metropolitan area networks: Frame Replication and Elimination for Reliability", DOI 10.1109/IEEESTD.2017.8091139, October 2017. https://standards.ieee.org/ieee/802.1CB/5703/

IEEE 802.1CBcv-2021: "Frame Replication and Elimination for Reliability, Amendment 1: Information Model, YANG Data Model, and Management Information Base Module", DOI 10.1109/IEEESTD.2022.9715061, February 2022. https://standards.ieee.org/ieee/802.1CBcv/7285/

IEEE 802.1Q-2022: "IEEE Standard for Local and Metropolitan Area Networks: Bridges and Bridged Networks", December 2022. https://standards.ieee.org/ieee/802.1Q/10323/

IEEE 802.1AS-2020: "IEEE Standard for Local and Metropolitan Area Networks: Timing and Synchronization for Time-Sensitive Applications". https://standards.ieee.org/ieee/802.1AS/7121/

IEEE 802.1Qbv-2015: "Bridges and Bridged Networks, Amendment 25: Enhancements for Scheduled Traffic". https://standards.ieee.org/ieee/802.1Qbv/6068/

12.3 Open-source implementations

DNT (Ericsson Research), Deterministic Networking Toolkit. https://github.com/EricssonResearch/dnt

DNT scenario_tsn, Layer 2 TSN and FRER reference scenario. https://github.com/EricssonResearch/dnt/blob/main/getting_started/scenario_tsn/README.md

DNT scenario_tsn_over_detnet, TSN over DetNet reference scenario. https://github.com/EricssonResearch/dnt/blob/main/getting_started/scenario_tsn_over_detnet/README.md

xdpfrer (Ericsson Research), FRER and PREF implementation using XDP and eBPF. https://github.com/EricssonResearch/xdpfrer

12.4 Emulation, routing, and analysis tooling

munet (LabN Consulting), a package for creating network topologies and running programs within them. https://github.com/LabNConsulting/munet

munet documentation, configuration and node reference. https://munet.readthedocs.io/en/latest/

mutest, munet test framework and user API. https://munet.readthedocs.io/en/latest/mutest.html

FRRouting project documentation. https://docs.frrouting.org

FRRouting Developer's Guide, "Building FRR for Ubuntu 24.04". https://docs.frrouting.org/projects/dev-guide/en/latest/building-frr-for-ubuntu2404.html

FRRouting Developer's Guide, "Topology Tests". https://docs.frrouting.org/projects/dev-guide/en/latest/topotests.html

libyang (CESNET), YANG data modeling language library. https://github.com/CESNET/libyang

tshark, terminal-based Wireshark network analyzer. https://www.wireshark.org/docs/man-pages/tshark.html

bridge(8), iproute2, show and manipulate bridge addresses and devices. https://man7.org/linux/man-pages/man8/bridge.8.html

12.5 Working group

IETF Deterministic Networking (DetNet) Working Group. https://datatracker.ietf.org/wg/detnet/about/